Interview Questions for idiots

- Front-End
  - What's the difference between localStorage, Cookie, SessionStorage?
  - What is the maximum size of cookie?
- JavaScript
  - Explain the differences between observables and promises.
  - What is closure?
- OOP
  - What is an Abstract Class?
  - How can you describe object-oriented concepts in detail?
  - Access modifiers?
- .Net Core
  - What are the features provided by ASP.NET Core?
  - What are the advantages of ASP.NET Core over ASP.NET?
  - What is the startup class in ASP.NET core?
  - What is Middleware?
  - What is the use of the ConfigureServices method of startup class?
  - What is the use of the Configure method of startup class?
  - What is the difference between IApplicationBuilder.Use() and IApplicationBuilder.Run()?
  - How to enable Session in ASP.NET Core?
  - What are services lifetime in asp core?
- Angular
  - What are Lifecycle hooks in Angular? Explain some life cycles hooks
  - Could we make an angular application to render on the server-side?
  - Explain Dependency Injection?
- SQL
  - What is the difference between View and Stored-Procedure?
  - Write a sample code for transaction block
  - Write a sample code for try ... catch
  - What happens if a transaction timeout and you want to rollback?
  - What is a trigger?
  - What are built-in functions and user defined functions?
  - Is it possible to use function in a view?
  - Write a sample code for fetching top 1 salary record from a table
  - Is it possible to send email by SQL?
  - Is it possible to access data to different databases from different servers (2008 and 2016) inside a single Stored-Procedure? If yes, how?
  - What is Normalization in SQL?
- C#

What is the difference between a delegate and an event in C#?
What is a covariance and contravariance in C#?
What is the difference between a static class and a singleton class in C#?
What is a closure in C#, and how does it work
What happens when an exception is thrown in C#?
What is garbage collection in C#?
What are the types of classes in C#?
What is the difference between Class and Structure in C#?
What is Polymorphism in C#? What are the different types of Polymorphism?
What is the difference between Early Binding and Late Binding in C#? or What is the difference between Compile Time Polymorphism and Run-Time Polymorphism in C#?
What is the difference between Method Overriding and Method Hiding?
What is the difference between a virtual method and an abstract method?
What is the difference between an abstract class and an interface?
What are extension methods in C#?
What are Generics in C#?
What are partial classes in C#?
What are Partial Methods in C#?
What are delegates in C# and the uses of delegates?
Why Do We Need Delegates?
What are sealed classes in C#?
What is Multithreading with .NET?
Explain the concept of thread in C#.
Define what multi-threaded execution means, and what concurrent execution means. Can they be used together?
Thread vs process
Parallel vs asynchronous
Don't mix Asynchronous and Multi-Threaded.
What is Reflection in C#?
What is the difference between constants and read-only?
What is Common Language Runtime (CLR)?
What is the JIT compiler process?
What is garbage collection in C#?
What is Managed or Unmanaged Code?
What is the difference between to dispose and finalize methods in C#?
What is the Constructor Chaining in C#?
What is a Virtual Method in C#?
What are Events?
Compare Virtual methods and Abstract methods.
What do you understand about Value types and Reference types in .NET? Provide some comparison.
What is the difference between string and StringBuilder in C#?
What is Managed or Unmanaged Code?

Intermediate Language & execution
IDisposable
Using HTTP Methods for RESTful Services
Describe the differences between Stack and Heap.
Explain how the runtime handles asynchronous calls -- walk me through the process.
Walk me through the different stages of garbage collection and how they're triggered.
Tell me about the most common search algorithms and how they function?
Describe how sorting algorithms work.
What tools and processes do you use for debugging? Why?
How would you debug a slow response time from an API call?
How would you solve an issue with a massive list of result sets on a web page?
Explain the differences between override and overload in object-oriented programming.
What's the purpose behind inversion of control (IoC) and dependency injection (DI)?
Describe what a Lambda is and how it would be used.
Have you done work with containerization?
Which clouds do you have the most experience with? What did you do?
Walk us through your experience with orchestration (Kubernetes, Docker Swarm, etc.).
What is your favorite programming language and why?
What are your biggest pet peeves about how other people code?
Common Type System & Common Language Specification
What is .Net Standard?
What is volatile in C#?
Difference between POST and PUT
Difference between PUT and PATCH
What is the http HEAD Method
What is cross-site request forgery (CSRF)?
What is Pass by Value and What is Pass by Reference?
What is the difference between .NET and .NET Core?
What is heap and stack?
When we call a generic method, where is T stored?
How can we overwrite a parent class method?
What is the difference between Graphql and RestApi?
What is polymorphism in OOP?
How do you handle your customer's new requirements?
What is UML and why do we use it?
What is Global Assembly Cache?
What is serialization and why do we use them?
Write a sample code for serialize an object
What is the difference between DataReader and DataSet?
Write a sample code for using a transaction by ADO.NET
When do we use MVC, MVP, and MVVM?
How could we return different views for a single controller?

- What is https?
- Why do we need WebApi?
- What have you used of SDLC method in your last project?
- Static Constructors
- Why would you use an orchestrator(Docker)?

SOLID
- S: Single Responsibility Principle (SRP)
- O: Open/Closed Principle
- L: Liskov Substitution Principle
- I: Interface Segregation Principle (ISP)
- D: Dependency Inversion Principle

Git
- What is the difference between Git vs SVN?
- What is the difference between git pull and git fetch?
- What is Git fork? What is the difference between fork, branch and clone?
- What is the difference between Rebase and Merge?
- Could you explain the Gitflow workflow?
- How to revert previous commit in git?
- What is git cherry-pick?
- Can you explain what git reset does in plain English?
- What is the difference between git clone, git clone --bare and git clone --mirror?
- How to undo the most recent commits in Git?
- What git command do you need to use to know who changed certain lines in a specific file?

React
- Why is isMounted() an anti-pattern and what is the proper solution?
- Can you force a component to re-render without calling setState?
- What is the difference between super() and super(props) in React using ES6 classes?
- What is the difference between setState() and replaceState() methods?
- What is Redux Thunk?
- Unidirectional Data Flow in React
- What are Higher-Order components?
- What is the purpose of using super constructor with props argument?
- What's the difference between a Controlled component and an Uncontrolled one in React?
- What are React Hooks?
- What are advantages of using React Hooks?
- What are the lifecycle methods of ReactJS?
- What is the difference between ShadowDOM and VirtualDOM?

Software Architectural Patterns:

Design Patterns
- Creational Design Patterns
- Structural Design Patterns

- Behavioral Design Patterns
- Microservices
  - Advantages of microservices:
  - What are the benefits of MicroServices?
  - How to separate monolith into microservices? How do you know something needs to be a microservice?
  - What are the disadvantages of microservices?
  - What is Saga design pattern?
- Rebus Configuration
- Creating The Saga With Rebus
- Message Types And Naming Conventions
- Saga Orchestration With Messages
- Handling Commands With Rebus
- In Summary
  - What is MassTransit?
  - What does MassTransit add to the transport?
- Questions from my own interviews
  - Using block in c# and why it is used
  - Difference between GET and POST
  - What is the difference between IQueryable and IEnumerable?
  - IEnumerable vs List
  - What is ajax and its advantages
  - What is cross origin?
  - Left join vs inner join
  - What is index in sql?
  - How do stack overflow
  - How to prevent stack overflow?
  - What is method hiding/shadowing in c#?
  - Types of dependency injection?

Front-End

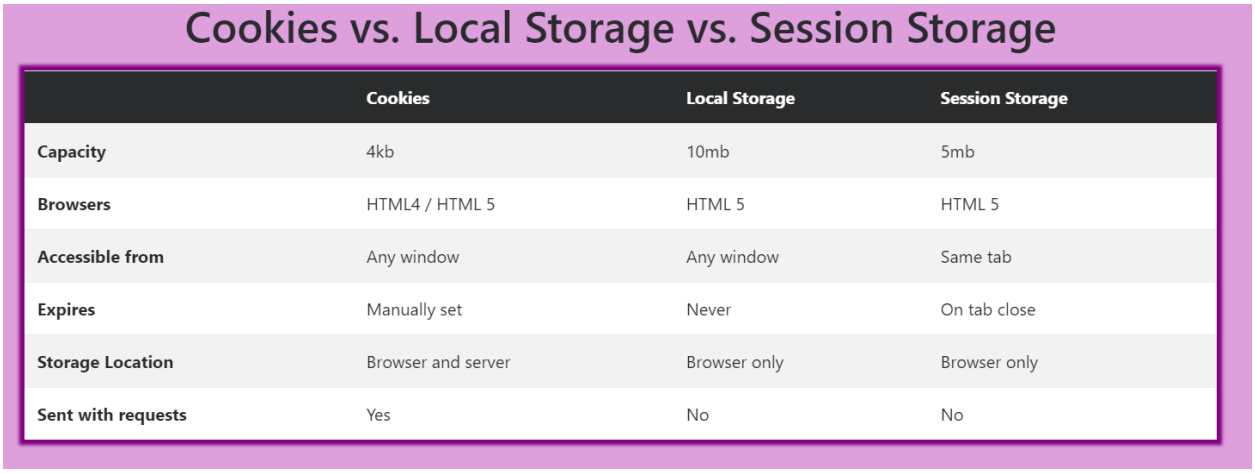
What's the difference between localStorage, Cookie, SessionStorage?

localStorage

1. data stored with localStorage has no expiration date, and gets cleared only through JavaScript, or clearing the Browser cache / Locally Stored Data.
2. The storage limit is the maximum amongst the three.
3. The data stored in localStorage persists until explicitly deleted. Changes made are saved and available for all current and future visits to the site.
4. It works on the same-origin policy. So, data stored will only be available on the same origin.

sessionStorage

1. It stores data only for a session, meaning that the data is stored until the browser (or tab) is closed.
2. Data is never transferred to the server.
3. Changes are only available per window (or tab in browsers like Chrome and Firefox). Changes made are saved and available for the current page, as well as future visits to the site on the same window. Once the window is closed, the storage is deleted.



	Cookies	Local Storage	Session Storage
Capacity	4kb	10mb	5mb
Browsers	HTML4 / HTML 5	HTML 5	HTML 5
Accessible from	Any window	Any window	Same tab
Expires	Manually set	Never	On tab close
Storage Location	Browser and server	Browser only	Browser only
Sent with requests	Yes	No	No

What is the maximum size of cookie?

4kb
300 cookies in total
20 cookies per domain

JavaScript

Explain the differences between observables and promises.

Observables

- Not executed until we subscribe to them using the subscribe() method.
- Are cancellable using the unsubscribe() method.
- Observable is a stream of values.

Promises:

- Execution starts right away.
- A Promise can provide a single value

What is closure?

Closure means that an **inner function** always has access to the **variables and parameters** of its **outer function**, even after the outer function has returned.
In other words, a closure gives you access to an outer function's scope from an inner function.
=> Closure remember the outer function scope even after creation time.

One important characteristic of closure is that outer variables can keep their states between multiple calls. Remember, inner function does not keep the separate copy of outer variables but it reference outer variables, that means value of the outer variables will be changed if you change it using inner function.

When to use Closure?
Closure is useful in hiding implementation detail in JavaScript. In other words, it can be useful to create private variables or functions.

Emulating private methods with closures
JavaScript, prior to classes, didn't have a native way of declaring private methods, but it was possible to emulate private methods using closures. Private methods aren't just useful for restricting access to code. They also provide a powerful way of managing your global namespace.
Read more…

OOP

What is an Abstract Class?

Abstract classes are classes that contain one or more abstract methods. An abstract method is a method that is declared, but contains no implementation. Abstract classes may not be instantiated, and require subclasses to provide implementations for the abstract methods.

How can you describe object-oriented concepts in detail?

C# is an object-oriented programming language that supports 4 OOP concepts.

1. Encapsulation

The word, “encapsulate,” means to enclose something. Just like a pill "encapsulates" or contains the medication inside of its coating, the principle of encapsulation works in a similar way in OOP: by forming a protective barrier around the information contained within a class from the rest of the code.

In OOP, we encapsulate by binding the data and functions which operate on that data into a single unit, the class. By doing so, we can hide private details of a class from the outside world and only expose functionality that is important for interfacing with it. When a class does not allow calling code access to its private data directly, we say that it is well encapsulated.

Example: *Elaborating on the person class example from earlier, we might have private data in the class, such as "socialSecurityNumber," that should not be exposed to other objects in the program. By encapsulating this data member as a private variable in the class, outside code would not have direct access to it, and it would remain safe within that person’s object.*

If a method is written in the person class to perform, say, a bank transaction called "bankTransaction()," that function could then access the "socialSecurityNumber"

variable as necessary. The person's private data would be well encapsulated in such a class.

2. Abstraction

Often, it's easier to reason and design a program when you can separate the interface of a class from its implementation, and focus on the interface. This is akin to treating a system as a "black box," where it's not important to understand the gory inner workings in order to reap the benefits of using it.

This process is called "abstraction" in OOP, because we are abstracting away the gory implementation details of a class and only presenting a clean and easy-to-use interface via the class' member functions. Carefully used, abstraction helps isolate the impact of changes made to the code, so that if something goes wrong, the change will only affect the implementation details of a class and not the outside code.

Example: *Think of a stereo system as an object with a complex logic board on the inside. It has buttons on the outside to allow for interaction with the object. When you press any of the buttons, you're not thinking about what happens on the inside because you can't see it. Even though you can't see the logic board completing these functions as a result of pressing a button, it's still performing them.*, albeit hidden to you.

This is the concept of abstraction, which is incredibly useful in all areas of engineering and also applied to great effect in object-oriented programming.

Example: *In OOP, we might have a class defined to represent the human body. One might define some functions as part of its publicly facing interface such as "walk()" or "eatFood()." Calling code could call these functions and remain completely oblivious to the complex inner workings of the human body and its necessary functions to perform the act of walking or eating. These details are completely hidden in the implementation of the walk() and eatFood() body functions and are, therefore, us abstracted away from*

the end user. In these cases, it's not important for calling code to understand how the brain coordinates walking or how the stomach manages digesting the food, but rather simply that a human walked or ate.

3. Inheritance

Object-oriented languages that support classes almost always support the notion of "inheritance." Classes can be organized into hierarchies, where a class might have one or more parent or child classes. If a class has a parent class, we say it is derived or inherited from the parent class and it represents an "IS-A" type relationship. That is to say, the child class "IS-A" type of the parent class.

Therefore, if a class inherits from another class, it automatically obtains a lot of the same functionality and properties from that class and can be extended to contain separate code and data. A nice feature of inheritance is that it often leads to good code reuse since a parent class' functions don't need to be re-defined in any of its child classes.

Consider two classes: one being the superclass—or parent—and the other being the subclass—or child. The child class will inherit the properties of the parent class, possibly modifying or extending its behavior. Programmers applying the technique of inheritance arrange these classes into what is called an "IS-A" type of relationship.

Example: *For instance, in the animal world, an insect could be represented by an Insect superclass. All insects share similar properties, such as having six legs and an exoskeleton. Subclasses might be defined for grasshoppers and ants. Because they inherit or are derived from the Insect class, they automatically share all insect properties.*

4. Polymorphism

In OOP, polymorphism allows for the uniform treatment of classes in a hierarchy. Therefore, calling code only needs to be written to handle objects from the root of the

hierarchy, and any object instantiated by any child class in the hierarchy will be handled in the same way.

Because derived objects share the same interface as their parents, the calling code can call any function in that class' interface. At run-time, the appropriate function will be called depending on the type of object passed leading to possibly different behaviors.

Example: *Suppose we have a class called, "Animal" and two child classes, "Cat," and "Dog." If the Animal class has a method to make a noise, called, "makeNoise," then, we can override the "makeNoise" function that is inherited by the sub-classes, "Cat" and "Dog," to be "meow" and "bark," respectively. Another function can, then, be written that accepts any Animal object as a parameter and invokes its "makeNoise" member function. The noise will be different: either a "meow" or a "bark" depending on the type of animal object that was actually passed to the function.*

Access modifiers?

Specifier	Same assembly			Other assembly	
	Declared class	other classes	Derived classes	other classes	Derived classes
Private	Yes	No	No	No	No
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	No	Yes	No	Yes
Internal	Yes	Yes	Yes	No	No
Protected Internal	Yes	Yes	Yes	No	Yes
Private Protected (C# 7.2)	Yes	No	Yes	No	No

.Net Core

What are the features provided by ASP.NET Core?

- Built-in supports for Dependency Injection
- Built-in supports for the logging framework and it can be extensible
- Introduced a new, fast and cross-platform web server - Kestrel. So, a web application can run without IIS, Apache, and Nginx.
- Provide protection against CSRF (Cross-Site Request Forgery)
- Multiple hosting ways are supported
- It supports modularity, so the developer needs to include the module required by the application. However, .NET Core framework is also providing the meta package that includes the libraries
- Command-line supports to create, build and run the application
- There is no web.config file. We can store the custom configuration into an appsettings.json file
- There is no Global.asax file. We can now register and use the services into startup class
- It has good support for asynchronous programming
- Support WebSocket and SignalR

What are the advantages of ASP.NET Core over ASP.NET?

- ASP.NET Core is cross platform, so you can run it on Windows, Linux and Mac.
- It can handle a lot more requests.
- You don't need to install the .NET Framework to run it. Instead, you ship all the required dlls with your application.
- You can use Visual Studio Code to develop your application, which is free and lightweight in comparison to Visual Studio. And you can use it on different platforms as well.
- A unified story for building web UI and web APIs.
- Architected for testability.
- Razor Pages makes coding page-focused scenarios easier and more productive.
- Blazor lets you use C# in the browser alongside JavaScript. Share server-side and client-side app logic all written with .NET.
- Ability to develop and run on Windows, macOS, and Linux.
- Open-source and community-focused.
- Integration of modern, client-side frameworks and development workflows.
- Support for hosting Remote Procedure Call (RPC) services using gRPC.
- A cloud-ready, environment-based configuration system.
- Built-in dependency injection.

- A lightweight, high-performance, and modular HTTP request pipeline.
- Ability to host on the following:
    - Kestrel
    - IIS
    - HTTP.sys
    - Nginx
    - Apache
    - Docker
- Side-by-side versioning.
- Tooling that simplifies modern web development.

.NET Core is written from scratch to make it a modular, lightweight, fast, and cross-platform Framework.

What is the startup class in ASP.NET core?

Startup class is the entry point of the ASP.NET Core application. Every .NET Core application must have this class.

The Startup class is where:

- Services required by the app are configured.
- The app's request handling pipeline is defined, as a series of middleware components.

Read more

What is Middleware?

The request handling pipeline is composed as a series of middleware components. Each component performs operations on an HttpContext and either invokes the next middleware in the pipeline or terminates the request.

By convention, a middleware component is added to the pipeline by invoking a Use... extension method in the Startup.Configure method. For example, to enable rendering of static files, call UseStaticFiles.

Each middleware component in the request pipeline is responsible for invoking the next component in the pipeline or short-circuiting the pipeline. When a middleware short-circuits, it's called a *terminal middleware* because it prevents further middleware from processing the request.

[*second answer:*
It is software which is injected into the application pipeline to handle requests and responses. They are just like chained to each other and form as a pipeline. The incoming requests are passed through this pipeline where all middleware is configured, and middleware can perform some action on the request before passing it to the next middleware. Same as for the responses, they are also passing through the middleware but in reverse order.
]

Read more

What is the use of the ConfigureServices method of startup class?

This is an optional method of startup class. It can be used to *configure the services that are used by the application*. This method calls first when the application is requested for the first time. Using this method, we can add the services to the DI container, so services are available as a dependency in the controller constructor.

What is the use of the Configure method of startup class?

It defines how the application will respond to each HTTP request. We can configure the request pipeline by configuring the middleware. It accepts IApplicationBuilder as a parameter and also it has two optional parameters: IHostingEnvironment and ILoggerFactory. Using this method, we can configure built-in middleware such as routing, authentication, session, etc. as well as third-party middleware.

What is the difference between IApplicationBuilder.Use() and IApplicationBuilder.Run()?

We can use both the methods in the Configure methods of startup class. Both are used to add a middleware delegate to the application request pipeline. The middleware added by using IApplicationBuilder.Use may call the next middleware in the pipeline whereas the middleware added using IApplicationBuilder.Run method never calls the subsequent or next middleware. After the IApplicationBuilder.Run method, the system stops adding middleware in the request pipeline.

How to enable Session in ASP.NET Core?

The middleware for the session is provided by the package Microsoft.AspNetCore.Session. To use the session in ASP.NET Core application, we need to add this package to csproj file and add the Session middleware to ASP.NET Core request pipeline.

What are services lifetime in asp core?

Singleton which creates a single instance throughout the application. It creates the instance for the first time and reuses the same object in all calls.

Scoped lifetime services are created once per request within the scope. It is equivalent to a singleton in the current scope. For example, in MVC it creates one instance for each HTTP request, but it uses the same instance in the other calls within the same web request.

Transient lifetime services are created each time they are requested. This lifetime works best for lightweight, stateless services.

Angular

What are Lifecycle hooks in Angular? Explain some life cycles hooks

Angular components enter its lifecycle from the time it is created to the time it is destroyed. Angular hooks provide ways to tap into these phases and trigger changes at specific phases in a lifecycle.

ngOnChanges(): This method is called whenever one or more input properties of the component changes. The hook receives a SimpleChanges object containing the previous and current values of the property.

ngOnInit(): This hook gets called once, after the ngOnChanges hook. It initializes the component and sets the input properties of the component.

ngDoCheck(): It gets called after ngOnChanges and ngOnInit and is used to detect and act on changes that cannot be detected by Angular. We can implement our change detection algorithm in this hook.

ngAfterContentInit(): It gets called after the first ngDoCheck hook. This hook responds after the content gets projected inside the component.

ngAfterContentChecked(): It gets called after ngAfterContentInit and every subsequent ngDoCheck. It responds after the projected content is checked.

ngAfterViewInit(): It responds after a component's view, or a child component's view is initialized.

ngAfterViewChecked(): It gets called after ngAfterViewInit, and it responds after the component's view, or the child component's view is checked.

ngOnDestroy(): It gets called just before Angular destroys the component. This hook can be used to clean up the code and detach event handlers.

Could we make an angular application to render on the server-side?

Yes, we can, with Angular Universal, a technology provided by Angular capable of rendering applications on the server-side.

The benefits of using Angular Universal are:

- **Better User Experience:** Allows users to see the view of the application instantly.
- **Better SEO:** Universal ensures that the content is available on every search engine leading to better SEO.
- **Loads Faster:** Render pages are available to the browsers sooner, so the server-side application loads faster.

Explain Dependency Injection?

Dependency injection is an application design pattern that is implemented by Angular and forms the core concepts of Angular.

Let us understand in a detailed manner. Dependencies in Angular are services which have a functionality. Various components and directives in an application can need these functionalities of the service. Angular provides a smooth mechanism by which these dependencies are injected into components and directives.

SQL

What is the difference between View and Stored-Procedure?

Stored Procedure : Stored procedures are precompiled database queries that improve the security, efficiency and usability of database client/server applications. Developers specify a stored procedure in terms of input and output variables. They then compile the code on the database platform and make it available to aplication developers for use in other environments, such as web applications. All of the major database platforms, including Oracle, SQL Server and MySQL support stored procedures. The major benefits of this technology are the substantial performance gains from precompiled execution, the reduction of client/server traffic, development efficiency gains from code reuse and abstraction and the security controls inherent in granting users permissions on specific stored procedures instead of the underlying database tables.

Views: Database views allow you to create "virtual tables" that are generated on the fly when they are accessed. A view is stored on the database server as an SQL statement that pulls data from one or more tables and (optionally) performs transformations on that data. Users may then query the view just as they would any real database table. Views are often used to alleviate security concerns by providing users with access to a certain view of a database table without providing access to the underlying table itself.

S.No.	**View**	**Stored Procedure**
1	Does not accepts parameters	Accept parameters
2	Can be used as a building block in large query.	Can not be used as a building block in large query.
3	Can contain only one single Select query.	Can contain several statement like if, else, loop etc.
4	Can not perform modification to any table.	Can perform modification to one or several tables.
5	Can be used (sometimes) as the target for Insert, update, delete queries.	Can not be used as the target for Insert, update, delete queries.

Write a sample code for transaction block

```
BEGIN TRAN
UPDATE Person
SET    Lastname = 'Lucky',
        Firstname = 'Luke'
WHERE  PersonID = 1
SELECT @@TRANCOUNT AS OpenTransactions
COMMIT TRAN
SELECT @@TRANCOUNT AS OpenTransactions
```

Write a sample code for try ... catch

```
BEGIN TRY
     BEGIN TRANSACTION;
     -- delete the person
     DELETE FROM sales.persons
     WHERE person_id = @person_id;
     -- if DELETE succeeds, commit the transaction
     COMMIT TRANSACTION;
 END TRY
 BEGIN CATCH
      ROLLBACK TRANSACTION;
 END CATCH
```

What happens if a transaction timeout and you want to rollback?

?

What is a trigger?

In SQL Server, triggers are database objects, actually, a special kind of stored procedure, which “reacts” to certain actions we make in the database.
A SQL trigger is a database object which fires when an event occurs in a database. We can execute a SQL query that will "do something" in a database when a change occurs on a database table such as a record is inserted or updated or deleted. For example, a trigger can be set on a record insert in a database table.

Types of Triggers

There are two types of triggers:

1. DDL Trigger
2. DML Trigger

DDL Triggers

The DDL triggers are fired in response to DDL (Data Definition Language) command events that start with Create, Alter and Drop, such as Create_table, Create_view, drop_table, Drop_view and Alter_table.

DML Triggers

The DML triggers are fired in response to DML (Data Manipulation Language) command events that start with Insert, Update, and Delete. Like insert_table, Update_view and Delete_table.

What are built-in functions and user defined functions?

Is it possible to use function in a view?

Yse you can.

Write a sample code for fetching top 1 salary record from a table

Is it possible to send email by SQL?

To send mail through SQL Server we need to set up DB mail profile we can either use T-SQl or SQL Database mail option in sql server to create profile.

Is it possible to access data to different databases from different servers (2008 and 2016) inside a single Stored-Procedure? If yes, how?

We can use linked servers.

What is Normalization in SQL?

Database Normalization or Data Normalization is the process of structuring a relational database to reduce data redundancy and improve data integrity. It was first proposed by Edgar F. Codd in 1970.

Normalization includes creating tables and establishing relationships between those tables according to normalization rules. Normalization is the process of reorganizing data in a database so that it meets two basic requirements:

- There is no redundancy of data, all data is stored in only one place.
- Data dependencies are logical, all related data items are stored together.

Normalization is a database design technique, which is used to design a relational database table up to a higher normal form. The process is progressive, and a higher level of database normalization cannot be achieved unless the previous levels have been satisfied.

There are a series of standard rules known as Normal Forms which are used to normalize the data. Multiple Normal forms are used based on the requirement to normalize the database.

A fully normalized database allows its structure to be extended to accommodate new types of data without changing the existing structure too much. As a result, applications interacting with the database are minimally affected.

Advantage of Normalization:

1. Normalization is the only way of organizing the data in the database.
2. Normalization divides the larger table into the smaller table and links them using a relationship so that operation can happen only on the required data table.
3. Normalization is used to minimize data redundancy.

4. Normalization is used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.

C#

What is the difference between a delegate and an event in C#?

A delegate is a type that represents a method signature, and can be used to pass methods as parameters to other methods or to store references to methods. An event is a mechanism that allows an object to notify other objects when something happens, such as when a button is clicked or a file is saved. An event is based on a delegate, and is essentially a list of methods that will be called when the event is raised

What is a covariance and contravariance in C#?

Covariance and contravariance are two concepts that relate to the compatibility of types in a type hierarchy. Covariance means that a type can be converted to a more derived type, while contravariance means that a type can be converted to a less derived type. In C#, covariance and contravariance are supported for interfaces, arrays, and delegate types. For example, if we have an interface called IAnimal and a class called Dog that implements IAnimal, we can use covariance to assign a List<Dog> to a variable of type List<IAnimal>.
https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/covariance-contravariance/

What is the difference between a static class and a singleton class in C#?

A static class is a class that cannot be instantiated and can only contain static members, which can be accessed without creating an instance of the class. A singleton class is a class that can only be instantiated once, and provides a global point of access to its instance. Static classes are often used to provide utility or helper methods that do not require state, while singleton classes are used to ensure that a single instance of a class is used throughout an application.

What is a closure in C#, and how does it work

A closure is a function that captures variables from its outer scope, allowing it to access those variables even after the outer function has returned. In C#, closures are commonly used with lambda expressions, which can capture variables from the surrounding code and use them in the lambda body. When a closure is created, a new instance of the function is created with a reference to the captured variables, allowing the closure to maintain access to those variables even after the surrounding function has completed.

https://www.infoworld.com/article/3620248/how-to-use-closures-in-csharp.html

What happens when an exception is thrown in C#?

When an exception is thrown in C#, the runtime searches the call stack for a catch block that can handle the exception. If no matching catch block is found, the exception is propagated up the call stack until it reaches the top-level entry point of the application, at which point the runtime will terminate the application and display an error message. The catch block can use the exception object to get information about the exception and perform any necessary cleanup or error handling.

What is garbage collection in C#?

Garbage collection is the process of freeing up memory that is captured by unwanted objects. When you create a class object, automatically some memory space is allocated to the object in the heap memory. Now, after you perform all the actions on the object, the memory space occupied by the object becomes waste. It is necessary to free up memory. Garbage collection happens in three cases:

- If the occupied memory by the objects exceeds the pre-set threshold value.
- If the garbage collection method is called
- If your system has low physical memory

What are the types of classes in C#?

Class is an entity that encapsulates all the properties of its objects and instances as a single unit. C# has four types of such classes:

- **Static class**: Static class, defined by the keyword ‘static’ does not allow inheritance. Therefore, you cannot create an object for a static class.
- **Partial class**: Partial class, defined by the keyword ‘partial’ allows its members to partially divide or share source (.cs) files.
- **Abstract class**: Abstract classes are classes that cannot be **instantiated** where you cannot create objects. Abstract classes work on the OOPS concept of abstraction. Abstraction helps to extract essential details and hide the unessential ones.

- **Sealed class**: Sealed classes are classes that cannot be inherited. Use the keyword sealed to restrict access to users to inherit that class.

What is the difference between Class and Structure in C#?

- All the structs are value types while classes are reference types.
- Structs cannot have destructors, but classes can have destructors.
- Structs cannot have an explicit default constructor while a class can have a default constructor.
- Structs do not support inheritance while classes support inheritance.
- Instance field declarations for a struct are not permitted to include variable initializers while a class can initialize the field members.
- In C# user can copy one structure object into another one using the assignment operator (=) but a class cannot be.
- A structure cannot have the abstract, virtual, or sealed members while a class can have.
- A structure cannot have a sealed method but a class can have sealed methods.
- A structure can only override the Object class virtual methods while a class can override any base class virtual method and abstract method.

What is Polymorphism in C#? What are the different types of Polymorphism?

Polymorphism is often referred to as the third pillar of object-oriented programming, after encapsulation and inheritance. The term "polymorphism" means "having many forms". So in OOPS, it means that an object can have multiple functionalities.

There are two types of polymorphism in OOPS.

1. Static Polymorphism:

Static Polymorphism or Early Binding involves linking a method with an object during compile time, hence also known as Compile Time Binding. Method Overloading and Operator Overloading are the way to achieve Static Polymorphism.

Please read the Static Polymorphism article here for more details. You can also read the Static Polymorphism Rules here.

2. Dynamic Polymorphism:

Dynamic Polymorphism or Late Binding involves linking a method with an object during run time, hence also known as Run Time-Binding. Method Overriding is a way to achieve Dynamic Polymorphism.

At run time, objects of a derived class may be treated as objects of a base class in places such as method parameters and collections or arrays. When this polymorphism occurs, the object's declared type is no longer identical to its run-time type.

What is the difference between Early Binding and Late Binding in C#? or What is the difference between Compile Time Polymorphism and Run-Time Polymorphism in C#?

In **Early Binding**, the compiler knows about what kind of object it is, what are all the methods and properties it contains. As soon as you declared the object, .NET Intellisense will populate its methods and properties. If a method or property does not exist or has data type problems then the compiler automatically throws an exception during compile time.

On the other hand in **Late Binding** compiler does not know what kind of object it is, what are all the methods and properties it contains. Everything will be known and loaded at the run time. When an object is dynamic or not known which will only bind during runtime is called Late binding.

Please read about the Early Binding here and Late Binding here for more details.

What is the difference between Method Overriding and Method Hiding?

1. Shadowing is used to protect against subsequent base class modification, while overriding does polymorphism by defining a different implementation.
2. Shadowing can redefine the entire signature, on the other hand, overriding can redefine only the implementation of a method, not the signature.
3. For overriding, the base class member must be declared as virtual while it is not mandatory for shadowing.
4. In shadowing, we can change the access modifier, while we cannot change the access modifier in overriding.
5. You cannot override a method with a property (or variable), or vice-versa, but you can shadow.
6. The "override" keyword cannot be used with the "virtual" modifier while the "new" keyword can be used with the "virtual" modifier.
7. In the case of shadowing, the base class cannot access the shadowed members in the derived class, while in the case of overriding the base class can access the overridden members in the derived class.
8. There is no control of a base class on shadowing i.e. a base class cannot enforce or prohibit shadowing, but it has control over overriding.
9. In overriding, a base class reference variable pointing to a child class object will invoke the overridden member in the derived class, not the base class. While in the case of shadowing it will invoke the base class members, not the derived class shadowed members.

What is the difference between a virtual method and an abstract method?

1. An abstract method cannot contain body definition while a virtual method must have a body definition.
2. Overriding all the abstract methods in the inheriting concrete class is mandatory while overriding the virtual methods is optional.
3. The "abstract" modifier can be used while overriding the abstract method but the "virtual" modifier cannot be used while overriding a virtual method.
4. An abstract method can only be declared in an abstract class while a virtual method can be defined in a non-abstract class as well.
5. A virtual method can be overridden to abstract for achieving re-abstraction but an abstract method cannot be overridden to virtual.

What is the difference between an abstract class and an interface?

Let's dig into the differences between an abstract class and an interface:

- Abstract classes are classes that cannot be instantiated ie. that cannot create an object. The interface is like an abstract class because all the methods inside the interface are abstract methods.
- Surprisingly, abstract classes can have both abstract and non-abstract methods but all the methods of an interface are abstract methods.
- Since abstract classes can have both abstract and non-abstract methods, we need to use the Abstract keyword to declare abstract methods. But in the interface, there is no such need.

An abstract class has constructors while an interface encompasses none.

```
public abstract class Shape{
    public abstract void draw();
}
—------------------------------
public interface Paintable{
    void paint();
```

```
}
```

What are extension methods in C#?

Extension methods help to add new methods to the existing ones. The methods that are added are static. At times, when you want to add methods to an existing class but don't perceive the right to modify that class or don't hold the rights, you can create a new static class containing the new methods. Once the extended methods are declared, bind this class with the existing one and see that these methods will be added to the existing one.

What are Generics in C#?

Generic means the general form, not specific. In C#, generic means not specific to a particular data type.
C# allows you to define generic classes, interfaces, abstract classes, fields, methods, static methods, properties, events, delegates, and operators using the type parameter and without the specific data type. A type parameter is a placeholder for a particular type specified when creating an instance of the generic type.

Generics introduces the concept of type parameters to .NET, which make it possible to design classes and methods that defer the specification of one or more types until the class or method is declared and instantiated by client code.

Generics in C# mean not linked to any specific data type. Generics reduce the load of using boxing, unboxing, and typecasting objects.

What are partial classes in C#?

Partial classes implement the functionality of a single class into multiple files. These multiple files are combined into one during compile time. The partial class can be created using the partial keyword.

You can easily split the functionalities of methods, interfaces, or structures into multiple files. You can even add nested partial classes.

If any of the parts are declared as abstract, then the entire type is considered as abstract or if any of the parts are declared as sealed, then the entire type is considered as sealed or if any of the parts inherit a class, then the entire type inherits that class.

Different parts of the partial class can specify different base interfaces and the final type implements all of the interfaces listed by all of the partial declarations. In the example below PartialClass needs to provide the implementation for both IEmployee and ICustomer interface methods

There are several situations when splitting a class definition is desirable

- When working on large projects, splitting a class over separate files allows multiple programmers to work on it simultaneously.
- When working with automatically generated source code, the code can be added to the class without having to recreate the source file. Visual Studio uses this approach when creating windows form, Web service wrapper code, and so on.

What are Partial Methods in C#?

A partial class or a struct can contain partial methods. A partial method is created using the same partial keyword. Let us understand partial methods with an example.

```
namespace PartialDemo
{
    partial class PartialClass
```

```
    {
        // Declaration of the partial method.
        partial void PartialMethod();

        // A public method calling the partial method
        public void PublicMethod()
        {
            Console.WriteLine("Public Method Invoked");
            PartialMethod();
        }
    }
}
```

Notice that the PartialMethod() definition has the partial keyword and does not have a body(implementation) only the signature. The implementation of a partial method is optional. If we don't provide the implementation, the compiler removes the signature and all calls to the method.

The implementation can be provided in the same physical file or in another physical file that contains the partial class. In this example, the partial PartialMethod() is invoked within the PublicMethod().

- Partial methods in C# are private by default and it is a compile-time error to include any access modifiers, including private. The following code will raise an error stating – A partial method cannot have access modifiers or the virtual, abstract, override, new, sealed, or extern modifiers.
- It is a compile-time error to include declaration and implementation at the same time for a partial method. The code below produces a compile-time error.
- A partial method return type must be void. Including any other return type is a compile-time error.

- A partial method must be declared within a partial class or partial struct. A non-partial class or struct cannot include partial methods. The signature of the partial method declaration must match with the signature of the implementation.
- A partial method can be implemented only once. Trying to implement a partial method more than once raises a compile-time error.

Read more…

What are delegates in C# and the uses of delegates?

A Delegate is an abstraction of one or more function pointers (as existed in C++; the explanation about this is out of the scope of this article). The .NET has implemented the concept of function pointers in the form of delegates. With delegates, you can treat a function as data. Delegates allow functions to be passed as parameters, returned from a function as a value and stored in an array. Delegates have the following characteristics:

- Delegates are derived from the System.MulticastDelegate class.
- They have a signature and a return type. A function that is added to delegates must be compatible with this signature.
- Delegates can point to either static or instance methods.
- Once a delegate object has been created, it may dynamically invoke the methods it points to at runtime.
- Delegates can call methods synchronously and asynchronously.

The delegate contains a couple of useful fields. The first one holds a reference to an object, and the second holds a method pointer. When you invoke the delegate, the

instance method is called on the contained reference. However, if the object reference is null then the runtime understands this to mean that the method is a static method. Moreover, invoking a delegate syntactically is the exact same as calling a regular function. Therefore, delegates are perfect for implementing callbacks.

Why Do We Need Delegates?

Historically, the Windows API made frequent use of C-style function pointers to create callback functions. Using a callback, programmers were able to configure one function to report back to another function in the application. So the objective of using a callback is to handle button-clicking, menu-selection, and mouse-moving activities. But the problem with this traditional approach is that the callback functions were not type-safe. In the .NET framework, callbacks are still possible using delegates with a more efficient approach. Delegates maintain three important pieces of information:

- The parameters of the method.
- The address of the method it calls.
- The return type of the method.

A delegate is a solution for situations in which you want to pass methods around to other methods. You are so accustomed to passing data to methods as parameters that the idea of passing methods as an argument instead of data might sound a little strange. However, there are cases in which you have a method that does something, for instance, invoking some other method. You do not know at compile time what this second method is. That information is available only at runtime, hence Delegates are the device to overcome such complications.

What are the Types of Delegates in C#?

The Delegates in C# are classified into two types such as

1. **Single Cast Delegate**
2. **Multicast Delegate**

If a delegate is used for invoking a single method then it is called a single cast delegate or unicast delegate. In other words, we can say that the delegates that represent only a single function are known as single cast delegates.

If a delegate is used for invoking multiple methods then it is known as the multicast delegate. Or the delegates that represent more than one function are called Multicast delegates.

What is Multicast Delegate in C#?

A Multicast Delegate in C# is a delegate that holds the references of more than one function. When we invoke the multicast delegate, then all the functions which are referenced by the delegate are going to be invoked. If you want to call multiple methods using a delegate then all the method signatures should be the same.

In the below example, we have created one delegate whose signature is the same as the two methods i.e. GetArea and GetPerimeter. Then we created the instance of delegate and bind the two methods using the += operator. Similarly, you can use the -= operator to remove a function from the delegate. Once we bind the two methods with the delegate instance and then when we call the delegate, both methods are going to be executed.

```
namespace MulticastDelegateDemo
{
    public delegate void RectangleDelete(double Width, double
Height);
    public class Rectangle
    {
```

```
        public void GetArea(double Width, double Height)
        {
            Console.WriteLine(@"Area is {0}", (Width * Height));
        }
        public void GetPerimeter(double Width, double Height)
        {
            Console.WriteLine(@"Perimeter is {0}", (2 * (Width +
Height)));
        }
        static void Main(string[] args)
        {
            Rectangle rect = new Rectangle();
            RectangleDelete rectDelegate = new
RectangleDelete(rect.GetArea);
            //RectangleDelete rectDelegate = rect.GetArea;

            //binding a method with delegate object
            // In this example rectDelegate is a multicast delegate.
You use += operator
            // to chain delegates together and -= operator to remove.

            rectDelegate += rect.GetPerimeter;
            rectDelegate(23.45, 67.89);

            Console.WriteLine();
            rectDelegate.Invoke(13.45, 76.89);

            Console.WriteLine();
            //Removing a method from delegate object
            rectDelegate -= rect.GetPerimeter;
            rectDelegate.Invoke(13.45, 76.89);

            Console.ReadKey();
        }
    }
}
```

Read more...

What are sealed classes in C#?

Sealed classes are used to restrict the inheritance feature of object-oriented programming. Once a class is defined as a sealed class, the class cannot be inherited. In C#, the sealed modifier is used to define a class as sealed. In Visual Basic .NET the Not Inheritable keyword serves the purpose of the sealed class. If a class is derived from a sealed class then the compiler throws an error. If you have ever noticed, structs are sealed. You cannot derive a class from a struct.

What is Multithreading with .NET?

Multithreading allows a program to run multiple threads concurrently. The real usage of a thread is not about a single sequential thread, but rather using multiple threads in a single program. Multiple threads running at the same time and performing various tasks are referred to as Multithreading. A thread is considered to be a lightweight process because it runs within the context of a program and takes advantage of the resources allocated for that program.

A single-threaded process contains only one thread while a multithreaded process contains more than one thread for execution.

—----------

Multi-threading is a process that contains multiple threads within a single process. Here each thread performs different activities. For example, we have a class and this call contains two different methods, now using multithreading each method is executed by a separate thread. So the major advantage of multithreading is it works simultaneously, which means multiple tasks execute at the same time. And also maximizing the utilization of the CPU because multithreading works on a time-sharing concept meaning each thread takes its own time for execution and does not affect the execution of another thread, this time interval is given by the operating system.

Explain the concept of thread in C#.

Answer: A thread can be defined as the execution flow of any program and defines a unique flow of control. You can manage these threads' execution time so that their execution does not overlap the execution of other threads and prevent deadlock or to maintain efficient usage of resources. Threads are lightweight programs that save the CPU consumption and increase the efficiency of the application. The thread cycle starts with the creation of the object of system.threading.thread class and ends when the thread terminates.

System.threading.thread class allows you to handle multiple threads and the first thread always runs in a process called the main thread. Whenever you run a program in C#, the main thread runs automatically.

Define what multi-threaded execution means, and what concurrent execution means. Can they be used together?

short answer: Concurrency is two lines of customers ordering from a single cashier (lines take turns ordering); Parallelism is two lines of customers ordering from two cashiers (each line gets its own cashier).

Concurrency is when two or more tasks can start, run, and complete in overlapping time periods. It doesn't necessarily mean they'll ever both be running at the same instant. For example, multitasking on a single-core machine.

Parallelism is when tasks literally run at the same time, e.g., on a multicore processor.

Concurrency vs Parallelism (Often confused and mixed)

Concurrency: Concurrency is about two or more tasks, different or parts of the same task sharing time slots with each other using a single cpu.

Parallelism: Multiple cpu's running different tasks exactly at the same time.

Thread vs process

- Process means a program is in execution, whereas thread means a segment of a process.
- A Process is not Lightweight, whereas Threads are Lightweight.
- A Process takes more time to terminate, and the thread takes less time to terminate.
- Process takes more time for creation, whereas Thread takes less time for creation.
- Process likely takes more time for context switching whereas as Threads takes less time for context switching.
- A Process is mostly isolated, whereas Threads share memory.
- Process does not share data, and Threads share data with each other.

Parallel vs asynchronous

When you run something asynchronously it means it is non-blocking, you execute it without waiting for it to complete and carry on with other things. Parallelism means to run multiple things at the same time, in parallel. Parallelism works well when you can separate tasks into independent pieces of work.

—----------

Asynchonous programming solves the problem of waiting around for an expensive operation to complete before you can do anything else. If you can get other stuff done while you're waiting for the operation to complete then that's a good thing. Example: keeping a UI running while you go and retrieve more data from a web service.

Parallel programming is related but is more concerned with breaking a large task into smaller chunks that can be computed at the same time. The results of the smaller chunks can then be combined to produce the overall result. Example: ray-tracing where the colour of individual pixels is essentially independent.
Read more….

Don't mix Asynchronous and Multi-Threaded.

1. Synchronous Single-Threaded. We have just a single thread running. We execute task Task_A, then some IO operation IO_A, then task Task_B, then I/O operation IO_B, then task T_C, the I/O operation IO_C. As we can see, this single main thread is blocked during the times that it's waiting for the I/O operations to finish, it can't do anything else!

2. Synchronous Multi-Threaded. In this example, we have 2 threads. Each thread is essentially a worker. So in this case, we're still waiting on the I/O operation to finish, regardless of the threads, but now, thankfully due to threads, the overall execution time is lower because the work is distributed among the threads.

3. Asynchronous Single-Threaded. We have just a single thread running. We execute Task_A, then some IO operation IO_A. The good news is that we aren't blocked by IO_A , we can continue whatever we want to do next, because IO_A will inform us when it's finished. So as soon as we started IO_A, we can immediately go onto the next task T_B.

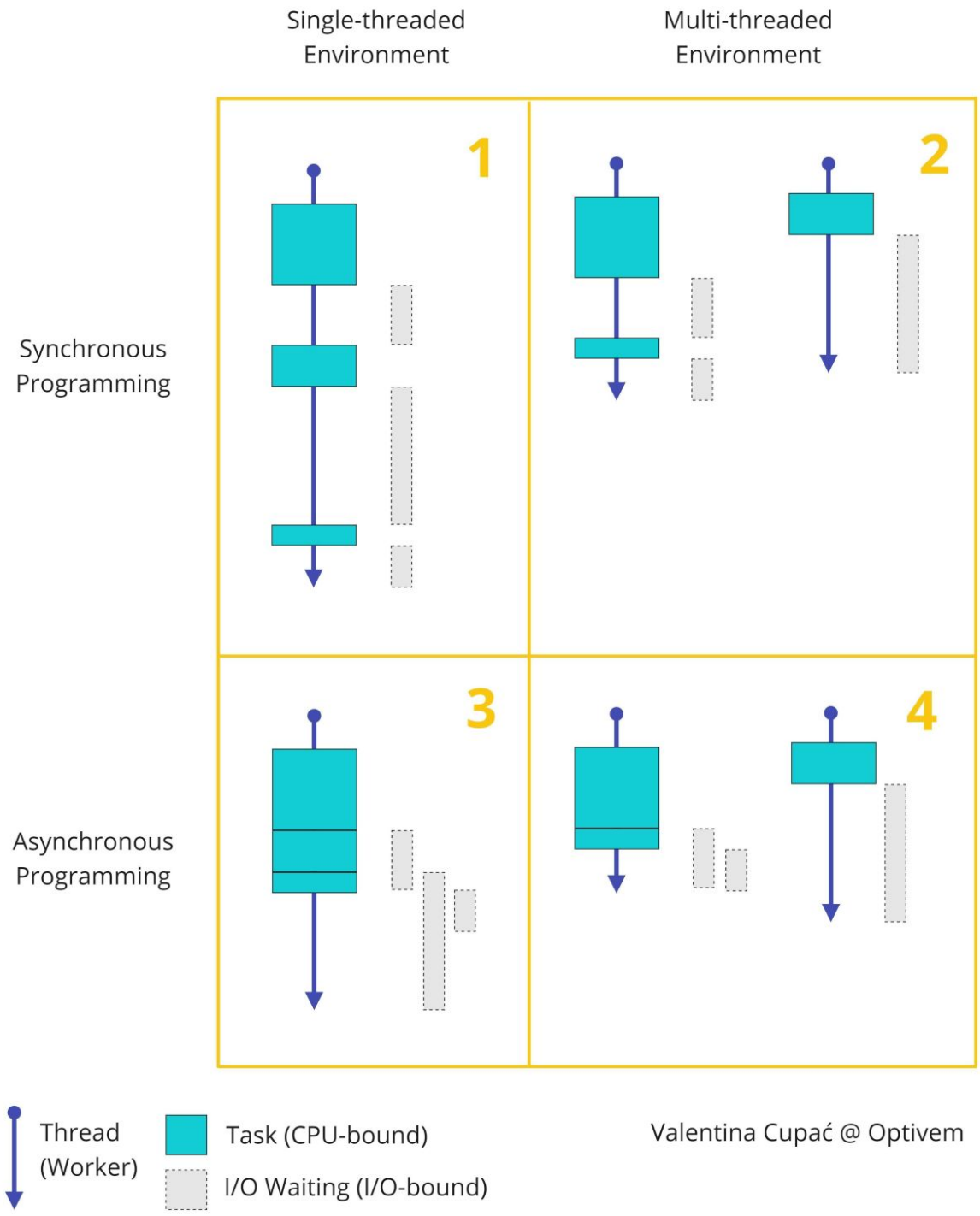
4. Asynchronous Multi-Threaded. In this example, we have 2 threads. Each thread is a worker. Now the great thing in this situation is that each thread, if it needs to perform some I/O work, the isn't blocked by the I/O work, but can go onto the next task.

To summarize:

- Synchronous versus Asynchronous. In Synchronous Programming, we're blocked by I/O operations, i.e. we can't do anything else in the meantime. In Asynchronous Programming, we're not blocked by I/O operations, i.e. we can proceed to go on to the next task. Asynchonous Programming is useful for avoiding blocking due to I/O waiting time.

- Single-Threaded versus Multi-Threaded. In Single-Threaded Environments, we have only one worker. In Multi-Threaded Environments, we have multiple workers. Multi-threading is useful for distributing CPU-bound work among workers.

Asynchronous Programming & Multi-threaded Environment



What is Reflection in C#?

Reflection is the process of runtime type discovery to inspect metadata, CIL code, late binding, and self-generating code. At the runtime by using reflection, we can access the same "type" information as displayed by the ildasm utility at design time. The reflection is analogous to reverse engineering in which we can break an existing *.exe or *.dll assembly to explore defined significant contents information, including methods, fields, events, and properties.

You can dynamically discover the set of interfaces supported by a given type using the System.Reflection namespace.

Reflection typically is used to dump out the loaded assemblies list, their reference to inspect methods, properties etcetera. Reflection is also used in the external disassembling tools such as Reflector, Fxcop, and NUnit because .NET tools don't need to parse the source code similar to C++.

Metadata Investigation

The following program depicts the process of reflection by creating a console-based application. This program will display the details of the fields, methods, properties, and interfaces for any type within the mscorlib.dll assembly. Before proceeding, it is mandatory to import "System.Reflection".

Here, we are defining a number of static methods in the program class to enumerate fields, methods, and interfaces in the specified type. The static method takes a single "System.Type" parameter and returns void.

```
static void FieldInvestigation(Type t) {
    Console.WriteLine("*********Fields*********");
    FieldInfo[] fld = t.GetFields();
    foreach(FieldInfo f in fld) {
        Console.WriteLine("-->{0}", f.Name);
    }
}
static void MethodInvestigation(Type t) {
    Console.WriteLine("*********Methods*********");
    MethodInfo[] mth = t.GetMethods();
    foreach(MethodInfo m in mth) {
        Console.WriteLine("-->{0}", m.Name);
    }
}
```

What is the difference between constants and read-only?

Constant variables are declared and initialized at compile time. The value can't be changed afterward. Read-only is used only when we want to assign the value at run time.

What is Common Language Runtime (CLR)?

CLR is the basic and Virtual Machine component of the .NET Framework. It is the run-time environment in the .NET Framework that runs the code and helps in making the development process easier by providing various services such as remoting, thread management, type-safety, memory management, robustness, etc. Basically, it is responsible for managing the execution of .NET programs regardless of any .NET programming language. It also helps in the management of code, as code that targets the runtime is known as the Managed Code, and code that doesn't target runtime is known as Unmanaged code.

What is the JIT compiler process?

Just-In-Time compiler(JIT) is a part of Common Language Runtime (CLR) in .NET which is responsible for managing the execution of .NET programs regardless of any .NET

programming language. A language-specific compiler converts the source code to the intermediate language. This intermediate language is then converted into the machine code by the Just-In-Time (JIT) compiler. This machine code is specific to the computer environment that the JIT compiler runs on.

What is garbage collection in C#?

Automatic memory management is made possible by Garbage Collection in .NET Framework. When a class object is created at runtime, certain memory space is allocated to it in the heap memory. However, after all the actions related to the object are completed in the program, the memory space allocated to it is a waste as it cannot be used. In this case, garbage collection is very useful as it automatically releases the memory space after it is no longer required.

Garbage collection will always work on Managed Heap, and internally it has an Engine which is known as the Optimization Engine. Garbage Collection occurs if at least one of multiple conditions is satisfied. These conditions are given as follows:

- If the system has low physical memory, then garbage collection is necessary.
- If the memory allocated to various objects in the heap memory exceeds a pre-set threshold, then garbage collection occurs.
- If the GC.Collect method is called, then garbage collection occurs. However, this method is only called under unusual situations as normally a garbage collector runs automatically.

What is Managed or Unmanaged Code?

A code that is written to get the services of the managed runtime environment execution like CLR(Common Language Runtime) in .NET Framework is known as Managed Code. It is always implemented by the managed runtime environment instead of directly executed by the operating system. The managed runtime environment provides different types of services like garbage collection, type checking, exception handling, bounds checking, etc. to code automatically without the interference of the programmer. It also provides memory allocation, type safety, etc to the code. The application is written in the languages like Java, C#, VB.Net, etc. is always aimed at runtime environment services to manage the execution, and the code written in these types of languages is known as managed code.

A code that is directly executed by the operating system is known as Unmanaged code. It is always aimed at the processor architecture and depends upon computer architecture. When this code is compiled it always tends to get a specific architecture and always runs on that platform, in other words, whenever you want to execute the same code for the different architecture you have to recompile that code again according to that architecture. It always compiles to the native code that is specific to the architecture. To read more, refer to the article: Managed and

What is the difference between to dispose and finalize methods in C#?

The primary difference between dispose() and finalize() is that the dispose() must be explicitly invoked by the user and the finalize() is called by the garbage collector when the object is destroyed.

What is the Constructor Chaining in C#?

We can call an overloaded constructor from another constructor using this keyword but the constructor must belong to the same class because this keyword is pointing to the members of the same class in which this is used. This type of calling the overloaded constructor is also termed as Constructor Chaining.

What is a Virtual Method in C#?

In C# a virtual method is a strategy that can be reclassified in derived classes. We can implement the virtual method in the base class and derived class. **It is utilized when a method's fundamental work is similar but in some cases derived classes need additional functionalities**. A virtual method is declared in the parent class that can be overridden in the child class. We make a virtual method in the base class by using the `virtual` keyword and that method is overridden in the derived class using the `Override` keyword. It is not necessary for every derived class to inherit a virtual method, but a virtual method must be created in the base class. Hence the virtual method is also known as Polymorphism.

What are Events?

An *event* is a notification that some action has occurred. Delegates and events are related because an event is built upon a delegate. Both expand the set of programming tasks to which C# can be applied. It is an important C# feature is built upon the foundation of delegates: the *event*. An event is, essentially, an automatic notification that some action has occurred. Events work like this:

An object that has an interest in an event registers an event handler for that event. When the event occurs, all registered handlers are called. Event handlers are represented by delegates.

Events are members of a class and are declared using the **event** keyword. Its most commonly used form is shown here:

event event-delegate event-name;

Here, *event-delegate* is the name of the delegate used to support the event, and *event-name* is the name of the specific event object being declared.

Compare Virtual methods and Abstract methods.

Answer: Any Virtual method must have a default implementation, and it can be overridden in the derived class using the override keyword. On the contrary, an Abstract method doesn't have an implementation, and it resides in the abstract class. The derived class must implement the abstract method. Though not necessary, we can use an override keyword here.

What do you understand about Value types and Reference types in .NET? Provide some comparison.

In C# data types can be of two types: Value Types and Reference Types. For a value type, the value is the information itself. For a reference type, the value is a reference which may be null or may be a way of navigating to an object containing the information.

- Value types - Holds some value not memory addresses. Example - Struct. A variable's value is stored wherever it is declared. Local variables live on the stack for example, but when declared inside a class as a member it lives on the heap tightly coupled with the class it is declared in.
  Advantages: A value type does not need extra garbage collection. It gets garbage collected together with the instance it lives in. Local variables in methods get cleaned up upon method leave.
  Drawbacks:
    i. When a large set of values are passed to a method the receiving variable actually copies so there are two redundant values in memory.
    ii. As classes are missed out.it losses all the oop benefits
- Reference type - Holds a memory address of a value not value. Example - Class. Stored on heap
  Advantages:

    i. When you pass a reference variable to a method and it changes it indeed changes the original value whereas in value types a copy of the given variable is taken and that value is changed.
    ii. When the size of variable is bigger reference type is good
    iii. As classes come as a reference type variables, they give reusability, thus benefiting Object-oriented programming
- Drawbacks: More work referencing when allocating and dereferences when reading the value.extra overload for garbage collector

What is the difference between string and StringBuilder in C#?

String

- It's an immutable object that holds string value.
- Performance wise string is slow because it creates a new instance to override or change the previous value.
- **`String` belongs to the `System` namespace.**

StringBuilder

- StringBuilder is a mutable object.
- Performance wise StringBuilder is very fast because it will use the same instance of StringBuilder object to perform any operation like insert value in existing string.
- `StringBuilder` belongs to `System.Text.Stringbuilder` namespace.

What is Managed or Unmanaged Code?

Managed code is just that: code whose **execution is managed by a runtime**. Runtime for .net languages is called the Common Language Runtime or CLR, regardless of the implementation (for example, Mono, .NET Framework, or .NET Core/.NET 5+). CLR is in charge of taking the managed code, compiling it into machine code and then executing

it. On top of that, runtime provides several important services such as automatic memory management, security boundaries, type safety etc.

Contrast this to the way you would run a C/C++ program, also called "**unmanaged code**". In the unmanaged world, the programmer is in charge of pretty much everything. The actual program is, essentially, a binary that the operating system (OS) loads into memory and starts. Everything else, from memory management to security considerations are a burden of the programmer.

Managed code is written in one of the high-level languages that can be run on top of .NET, such as C#, Visual Basic, F# and others. When you compile code written in those languages with their respective compiler, you don't get machine code. **You get Intermediate Language code** which the runtime then compiles and executes. C++ is the one exception to this rule, as it can also produce native, unmanaged binaries that run on Windows.

Intermediate Language & execution

What is "Intermediate Language" (or IL for short)? It is a product of compilation of code written in high-level .NET languages. Once you compile your code written in one of these languages, you will get a binary that is made out of IL. It is important to note that the IL is independent from any specific language that runs on top of the runtime; there is even a separate specification for it that you can read if you're so inclined.

Once you produce IL from your high-level code, you will most likely want to run it. This is where the CLR takes over and starts the process of Just-In-Time compiling, or JIT-ing your code from IL to machine code that can actually be run on a CPU. In this way, the CLR knows exactly what your code is doing and can effectively manage it.

Intermediate Language is sometimes also called Common Intermediate Language (CIL) or Microsoft Intermediate Language (MSIL).

IDisposable

The primary use of IDisposable interface is to release **unmanaged** resources. The garbage collector automatically releases the memory allocated to a managed object when that object is no longer used. However, it is not possible to predict when garbage collection will occur. Furthermore, the garbage collector has no knowledge of unmanaged resources such as window handles, or open files and streams.

Using HTTP Methods for RESTful Services

Post

The POST verb is most-often utilized to **create** new resources. In particular, it's used to create subordinate resources. That is, subordinate to some other (e.g. parent) resource. In other words, when creating a new resource, POST to the parent and the service takes care of associating the new resource with the parent, assigning an ID (new resource URI), etc.

On successful creation, return HTTP status 201, returning a Location header with a link to the newly-created resource with the 201 HTTP status.

POST is neither safe nor idempotent. It is therefore recommended for non-idempotent resource requests. Making two identical POST requests will most-likely result in two resources containing the same information.

GET

The HTTP GET method is used to **read** (or retrieve) a representation of a resource. In the “happy” (or non-error) path, GET returns a representation in XML or JSON and an HTTP response code of 200 (OK). In an error case, it most often returns a 404 (NOT FOUND) or 400 (BAD REQUEST).

According to the design of the HTTP specification, GET (along with HEAD) requests are used only to read data and not change it. Therefore, when used this way, they are considered safe. That is, they can be called without risk of data modification or corruption—calling it once has the same effect as calling it 10 times, or none at all. Additionally, GET (and HEAD) is idempotent, which means that making multiple identical requests ends up having the same result as a single request.

Do not expose unsafe operations via GET—it should never modify any resources on the server.

PUT

PUT is most-often utilized for **update** capabilities, PUT-ing to a known resource URI with the request body containing the newly-updated representation of the original resource.

However, PUT can also be used to create a resource in the case where the resource ID is chosen by the client instead of by the server. In other words, if the PUT is to a URI that contains the value of a non-existent resource ID. Again, the request body contains a resource representation. Many feel this is convoluted and confusing. Consequently, this method of creation should be used sparingly, if at all.

Alternatively, use POST to create new resources and provide the client-defined ID in the body representation—presumably to a URI that doesn't include the ID of the resource (see POST below).

On successful update, return 200 (or 204 if not returning any content in the body) from a PUT. If using PUT for create, return HTTP status 201 on successful creation. A body in the response is optional—providing one consumes more bandwidth. It is not necessary to return a link via a Location header in the creation case since the client already set the resource ID.

PUT is not a safe operation, in that it modifies (or creates) state on the server, but it is idempotent. In other words, if you create or update a resource using PUT and then make that same call again, the resource is still there and still has the same state as it did with the first call.

If, for instance, calling PUT on a resource increments a counter within the resource, the call is no longer idempotent. Sometimes that happens and it may be enough to document that the call is not idempotent. However, it's recommended to keep PUT requests idempotent. It is strongly recommended to use POST for non-idempotent requests.

PATCH

PATCH is used for **modify** capabilities. The PATCH request only needs to contain the changes to the resource, not the complete resource.

This resembles PUT, but the body contains a set of instructions describing how a resource currently residing on the server should be modified to produce a new version. This means that the PATCH body should not just be a modified part of the resource, but in some kind of patch language like JSON Patch or XML Patch.

PATCH is neither safe nor idempotent. However, a PATCH request can be issued in such a way as to be idempotent, which also helps prevent bad outcomes from collisions between two PATCH requests on the same resource in a similar time frame. Collisions from multiple PATCH requests may be more dangerous than PUT collisions because some patch formats need to operate from a known base-point or else they will corrupt the resource. Clients using this kind of patch application should use a conditional request such that the request will fail if the resource has been updated since the client last accessed the resource. For example, the client can use a strong ETag in an If-Match header on the PATCH request.

DELETE

DELETE is pretty easy to understand. It is used to **delete** a resource identified by a URI.

On successful deletion, return HTTP status 200 (OK) along with a response body, perhaps the representation of the deleted item (often demands too much bandwidth), or a wrapped response (see Return Values below). Either that or return HTTP status 204 (NO CONTENT) with no response body. In other words, a 204 status with no body, or the JSEND-style response and HTTP status 200 are the recommended responses.

HTTP-spec-wise, DELETE operations are idempotent. If you DELETE a resource, it's removed. Repeatedly calling DELETE on that resource ends up the same: the resource is gone. If calling DELETE say, decrements a counter (within the resource), the DELETE call is no longer idempotent. As mentioned previously, usage statistics and measurements may be updated while still considering the service idempotent as long as no resource data is changed. Using POST for non-idempotent resource requests is recommended.

There is a caveat about DELETE idempotence, however. Calling DELETE on a resource a second time will often return a 404 (NOT FOUND) since it was already removed and

therefore is no longer findable. This, by some opinions, makes DELETE operations no longer idempotent, however, the end-state of the resource is the same. Returning a 404 is acceptable and communicates accurately the status of the call.

Describe the differences between Stack and Heap.

Parameter	Stack	Heap
Type of data structures	A stack is a linear data structure.	Heap is a hierarchical data structure.
Access speed	High-speed access	Slower compared to stack
Space management	Space is managed efficiently by the OS so memory will never become fragmented.	Heap Space is not used as efficiently. Memory can become fragmented as blocks of memory first allocated and then freed.
Access	Local variables only	It allows you to access variables globally.
Limit of space size	Limit on stack size dependent on OS.	Does not have a specific limit on memory size.
Resize	Variables cannot be resized	Variables can be resized.
Memory Allocation	Memory is allocated in a contiguous block.	Memory is allocated in any random order.
Allocation and Deallocation	Automatically done by compiler instructions.	It is manually done by the programmer.
Deallocation	Does not require to deallocate variables.	Explicit deallocation is needed.
Cost	Less	More
Implementation	A stack can be implemented in 3 ways: simple array based, using dynamic memory, and Linked list based.	Heap can be implemented using arrays and trees.

Main Issue	Shortage of memory	Memory fragmentation
Locality of reference	Automatic compile time instructions.	Adequate
Flexibility	Fixed size	Resizing is possible
Access time	Faster	Slower

Explain how the runtime handles asynchronous calls -- walk me through the process.

??

Walk me through the different stages of garbage collection and how they're triggered.

??

Tell me about the most common search algorithms and how they function?

??

Describe how sorting algorithms work.

??

What tools and processes do you use for debugging? Why?

??

How would you debug a slow response time from an API call?

??

How would you solve an issue with a massive list of result sets on a web page?

??

Explain the differences between override and overload in object-oriented programming.

Method Overloading	Method Overriding
Method overloading is used *to increase the readability* of the program.	Method overriding is used *to provide the specific implementation* of the method that is already provided by its super class.
Method overloading is performed *within class*.	Method overriding occurs *in two classes* that have an IS-A (inheritance) relationship.
In case of method overloading, *parameters must be different*.	In case of method overriding, *parameters must be the same*.
Method overloading is the example of *compile time polymorphism*.	Method overriding is the example of *run time polymorphism*.
In Java, method overloading can't be performed by changing the return type of the method only. *Return type can be the same or different* in method overloading. But you must have to change the parameter.	*Return type must be same or covariant* in method overriding.

What's the purpose behind inversion of control (IoC) and dependency injection (DI)?

??

Describe what a Lambda is and how it would be used.

??

Have you done work with containerization?

??

Which clouds do you have the most experience with? What did you do?

??

Walk us through your experience with orchestration (Kubernetes, Docker Swarm, etc.).

??

What is your favorite programming language and why?

??

What are your biggest pet peeves about how other people code?

??

Common Type System & Common Language Specification

Read => https://docs.microsoft.com/en-us/dotnet/standard/common-type-system

What is .Net Standard?

.Net standard is a specification of .net APIs that any implementation of .net should implement.

If you want to share code between .NET Framework and any other .NET implementation, such as .NET Core, your library should target .NET Standard 2.0.

.NET 5, .NET 6, and all future versions will continue to support .NET Standard 2.1 and earlier.

.NET Standard is versioned. Each new version adds more APIs. When a library is built against a certain version of .NET Standard, it can run on any .NET implementation that implements that version of .NET Standard (or higher).

Targeting a higher version of .NET Standard allows a library to use more APIs but means it can only be used on more recent versions of .NET. Targeting a lower version reduces the available APIs but means the library can run in more places.

What is volatile in C#?

The `volatile` keyword indicates that a field might be modified by multiple threads that are executing at the same time. The compiler, the runtime system, and even hardware may rearrange reads and writes to memory locations for performance reasons. Fields that are declared `volatile` are excluded from certain kinds of optimizations. There is no guarantee of a single total ordering of volatile writes as seen from all threads of execution. For more information, see the Volatile class.

Difference between POST and PUT

The difference between POST and PUT is that PUT requests are idempotent. That is, calling the same PUT request multiple times will always produce the same result. In contrast, calling a POST request repeatedly have side effects of creating the same resource multiple times.

Difference between PUT and PATCH

The main difference between PUT and PATCH in REST API is that PUT handles updates by replacing the entire entity, while PATCH only updates the fields that you give it.

PATCH does not change any of the other values.

If you use the PUT method, then everything will get updated. Depends on server implementation, but in most REST APIs, this means it will overwrite any missing fields to null.

PATCH method updates individual fields without overwriting existing fields.

Use HTTP PUT when you are updating an entity that you cannot separate into smaller chunks. For example, we should update an avatar image as a complete entity, rather than patching the individual pixel bits.

PUT needs more bandwidth because it handles full resources, so PATCH was introduced to reduce the bandwidth needed.

In both cases, the HTTP response code should tell you whether the update is successful.

PUT

The HTTP PUT request method is used to create a new resource or replace an old one.

The IETF has defined the HTTP PUT method as "only allowing a full replacement of a document."

The client must send a complete entity in a request body with all the new values for the resource.

- client provides the resource ID
- if the resource exists, replace it with the inbound data
- if it doesn't exist, create a new one

Advantage of using PUT

An advantage of using PUT is that we don't have to re-fetch the entity after successfully updating it. Since PUT sends the entire entity, it'll replace whatever is there with what we send.

Another advantage of using PUT is that we can always overwrite the complete entity if we send down all fields and do not include any delta modification to existing fields. This means that our API clients only need to implement a single PUT method instead of many for partial updates.
PUT creates a resource if one doesn't exist, which can come useful.

PATCH
PATCH is used to change or add data on existing resources.
HTTP PUT method creates the entire resource at once, whereas PATCH only updates the fields that are being patched.

HTTP PATCH method was added to the HTTP protocol in 2010 to overcome the limitations of HTTP PUT request, which provides no support for partial updates.

Advantage of using PATCH

- The client doesn't need to know the complete state of the resource before sending an update request.
- PATCH makes it easier to keep a track of update intent to see what changed with each request. This advantage can be useful in situations where we need to review logs to debug. Noticing how resource change is easier with PATCH rather than PUT because the request body has very specific intent.
- PATCH saves bandwidth by allowing partial updates. It is more efficient than HTTP PUT for updating large objects.

Read more…

What is the http HEAD Method

HEAD is almost identical to GET, but without the response body.

In other words, if GET /users returns a list of users, then HEAD /users will make the same request but will not return the list of users.

HEAD requests are useful for checking what a GET request will return before actually making a GET request - like before downloading a large file or response body.

—--

The HTTP HEAD method requests the headers that would be returned if the HEAD request's URL was instead requested with the HTTP GET method. For example, if a URL might produce a large download, a HEAD request could read its Content-Length header to check the filesize without actually downloading the file.

Note: If the response to a HEAD request shows that a cached URL response is now outdated, the cached copy is invalidated even if no GET request was made.

What is cross-site request forgery (CSRF)?

??…….

What is Pass by Value and What is Pass by Reference?

Call By Value in C#:

- In Pass by Value, the copy of the original variable is passed to the called function.
- In Call by Value, the value of the original parameter is copied into the parameter of the function. As a result, if we do any modifications to formal parameters, they don't have any impact on the actual parameter.
- It will not allow you to change the actual variables using function calls.
- The memory location referred to by formal parameters and actual arguments is different.
- It doesn't require a ref or out keyword in C#.

Call By Reference in C#:

- In Pass by Reference, the reference of the variable is passed to the called function.
- In Call by Value, the argument and the given parameter both point to the same address. As a result, any changes made to any of the function's arguments are reflected in the passed parameter.
- It allows you to change the actual variables using function calls.
- The memory location referred to by formal parameters and actual arguments are the same
- It requires a ref or out keyword to achieve call by reference in C#.

Read more…

What is the difference between .NET and .NET Core?

???

What is heap and stack?

The stack is the memory set aside as scratch space for a thread of execution. When a function is called, a block is reserved on the top of the stack for local variables and some bookkeeping data. When that function returns, the block becomes unused and can be used the next time a function is called. The stack is always reserved in a LIFO (last in first out) order; the most recently reserved block is always the next block to be freed. This makes it really simple to keep track of the stack; freeing a block from the stack is nothing more than adjusting one pointer.

The heap is memory set aside for dynamic allocation. Unlike the stack, there's no enforced pattern to the allocation and deallocation of blocks from the heap; you can allocate a block at any time and free it at any time. This makes it much more complex to keep track of which parts of the heap are allocated or free at any given time; there are many custom heap allocators available to tune heap performance for different usage patterns.

Each thread gets a stack, while there's typically only one heap for the application (although it isn't uncommon to have multiple heaps for different types of allocation).

What is their scope?
The stack is attached to a thread, so when the thread exits the stack is reclaimed. The heap is typically allocated at application startup by the runtime, and is reclaimed when the application (technically process) exits.

What determines the size of each of them?
The size of the stack is set when a thread is created. The size of the heap is set on application startup, but can grow as space is needed (the allocator requests more memory from the operating system).
More:
https://www.programmerinterview.com/data-structures/difference-between-stack-and-heap
https://dotnettutorials.net/lesson/stack-and-heap-dotnet/

When we call a generic method, where is T stored?

Generics are stored as IL including the parameters and constraints in the metadata. Generic types are types as well, which are converted to runtime types at runtime. This is the main difference from C++ where the template types don't exist, but only the derived types; in C# the generic types exist in the DLL, but the derived types do not (until they are created at run-time).

How can we overwrite a parent class method?

The `override` modifier is required to extend or modify the abstract or virtual implementation of an inherited method, property, indexer, or event.

An `override` method provides a new implementation of the method inherited from a base class. The method that is overridden by an `override` declaration is known as the overridden base method. An `override` method must have the same signature as the overridden base method. Beginning with C# 9.0, `override` methods support covariant return types. In particular, the return type of an `override` method can derive from the return type of the corresponding base method. In C# 8.0 and earlier, the return types of an `override` method and the overridden base method must be the same.

You cannot override a non-virtual or static method. The overridden base method must be `virtual`, `abstract`, or `override`.

An `override` declaration cannot change the accessibility of the `virtual` method. Both the `override` method and the `virtual` method must have the same access level modifier.

You cannot use the `new`, `static`, or `virtual` modifiers to modify an `override` method.

An overriding property declaration must specify exactly the same access modifier, type, and name as the inherited property. Beginning with C# 9.0, read-only overriding properties support covariant return types. The overridden property must be `virtual`, `abstract`, or `override`.

What is the difference between Graphql and RestApi?

تفاوت اصلی این هست که خروجی api برای گرف میتونه توسط کلاینت کاستومایز بشه، در حالی که توی رست، خروجی یکیه.

What is polymorphism in OOP?

Polymorphism is one of the most important concepts of object oriented programming language.

The word “polymorphism” itself means “many forms” and it refers to when a class can take any form of class in its parent hierarchy.
For example we can create a subclass that has a different implementation for a method of its parent. Or a class that is implementing an interface can act however that it wants and different from other implementations.
—----------

Polymorphism is one of the most important concepts of object oriented programming language. The most common use of polymorphism in object-oriented programming occurs when a parent class reference is used to refer to a child class object. Here we will see how to represent any function in many types and many forms.

This means any child class object can take any form of a class in its parent hierarchy and of course itself as well. What this actually means is that the child class object can be assigned to any class reference in its parent hierarchy and of course itself as well.

How do you handle your customer's new requirements?

What is UML and why do we use it?

UML, short for Unified Modeling Language, is a standardized modeling language consisting of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems. The UML uses mostly graphical notations to express the design of software projects. Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

The primary goals in the design of the UML summarize by Page-Jones in Fundamental Object-Oriented Design in UML as follows:

- Provide users with a ready-to-use, expressive visual modeling language so they can develop and exchange meaningful models.
- Provide extensibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development processes.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of the OO tools market.
- Support higher-level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

What is Global Assembly Cache?

Don't know.

What is serialization and why do we use them?

Serialization is the process of **converting an object into a stream of bytes to store the object or transmit** it to memory, a database, or a file. Its main purpose is to save the state of an object in order to be able to recreate it when needed. The reverse process is called deserialization.

Write a sample code for serialize an object

```
Product product = new Product();
string output = JsonConvert.SerializeObject(product);
Product deserializedProduct = JsonConvert.DeserializeObject<Product>(output);
```

What is the difference between DataReader and DataSet?

In C#, we work in two different ways with database connectivity. As you know when we create software or a website or any other application which is connected to the database we need to make a connection to the database to get data. Here you can get data in two different ways.

Either it can be connected architecture where you go and connect to the database and get data or disconnected architecture where you connect to the database for the first time and get all data in an object and use it if required. You can perform any action like insert, update, and search on this.

You can use two C# objects to achieve this, first one is **DataSet** and other one is **DataReader**.

DataSet

It is a type of disconnected architecture. Disconnected architecture means, you don’t need to connect always when you want to get data from the database. You can get data

from dataset; basically DataSet is a collection of datatables. We can store the database table, view data in the DataSet and can also store the xml value in dataset and get it if required.

To achieve this you need to use **DataAdapter** which work as a mediator between Database and **DataSet**.

DataReader

It is a connected architecture, which means when you require data from the database you need to connect with database and fetch the data from there. You can use if you need updated data from the database in a faster manner. DataReader is Read/Forward only that means we can only get the data using this but we cannot update or insert the data into the database. It fetches the record one by one.

Write a sample code for using a transaction by ADO.NET

When do we use MVC, MVP, and MVVM?

How could we return different views for a single controller?

What is https?

It is Secure http. Which means your data that is transferred from your browser to the server is encrypted and no one can decrypt it but the server.

Why do we need WebApi?

??

What have you used of SDLC method in your last project?

Means The Software Development Life Cycle;
6 Software Development Life Cycle models

1. Agile
2. Lean
3. Waterfall
4. Iterative
5. Spiral
6. DevOps

Agile
The Agile model first emerged in 2001 and has since become the de facto industry standard. Some businesses value the Agile methodology so much that they apply it to other types of projects, including nontech initiatives.

In the Agile model, fast failure is a good thing. This approach produces ongoing release cycles, each featuring small, incremental changes from the previous release. At each iteration, the product is tested. The Agile model helps teams identify and address small issues on projects before they evolve into more significant problems, and it engages business stakeholders to give feedback throughout the development process.

As part of their embrace of this methodology, many teams also apply an Agile framework known as Scrum to help structure more complex development projects. Scrum teams work in sprints, which usually last two to four weeks, to complete assigned tasks. Daily Scrum meetings help the whole team monitor progress throughout the project. And the ScrumMaster is tasked with keeping the team focused on its goal.

Static Constructors

??

Cqrs

Why would you use an orchestrator(Docker)?

There are many benefits to using an orchestrator with a containerized microservice architecture.

The orchestrator helps with composing applications consisting of many microservices into one deployable unit. That unit is then moved — or deployed — to a host.

Once deployed, the orchestrator then helps with managing the host. It can automatically start the containers, scale them out with multiple instances per image, suspend them, or shut them down when needed. The orchestrator can also control how containers access resources like the network and data storage.

Orchestrators can perform tasks such as load-balancing and routing in scenarios when multiple containers exist on multiple hosts in a complex distributed system. They can also monitor the containers' and hosts' health.

SOLID

S: Single Responsibility Principle (SRP)

Every software module should have only one reason to change

This means that every class, or similar structure, in your code should have only one job to do. Everything in that class should be related to a single purpose.

O: Open/Closed Principle

A software module/class should be open for extension and closed for modification.

????

L: Liskov Substitution Principle

You should be able to use any derived class instead of a parent class and have it behave in the same manner without modification

It ensures that a derived class does not affect the behavior of the parent class, in other words, that a derived class must be substitutable for its base class.

I: Interface Segregation Principle (ISP)

Clients should not be forced to implement interfaces they don't use. Instead of one fat interface, many small interfaces are preferred based on groups of methods, each one serving one submodule

Like classes, each interface should have a specific purpose/responsibility (refer to SRP). You shouldn't be forced to implement an interface when your object doesn't share that purpose. The larger the interface, the more likely it includes methods that not all implementers can do.

D: Dependency Inversion Principle

High-level modules/classes should not depend on low-level modules/classes. Both should depend upon abstractions.

A high-level module/class that has a dependency on low-level modules/classes or some other class and knows a lot about the other classes it interacts with is said to be tightly coupled. When a class knows explicitly about the design and implementation of another class, it raises the risk that changes to one class will break the other class. So we must keep these high-level and low-level modules/classes loosely coupled as much as we can. To do that, we need to make both of them dependent on abstractions instead of knowing each other.

Git

What is the difference between Git vs SVN?

Git is a distributed version control system (DVCS), whereas SVN is a centralized version control system.

What is the difference between git pull and git fetch?

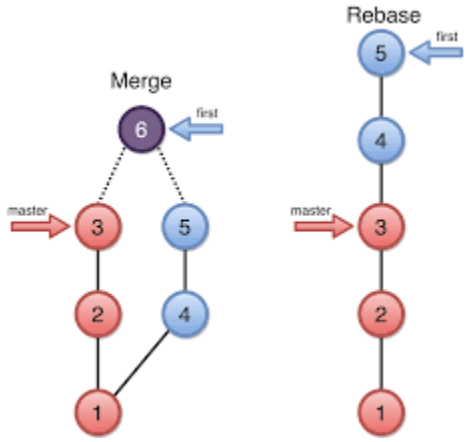
In the simplest terms, git pull does a git fetch followed by a git merge.
When you use pull, Git tries to automatically do your work for you. It is context sensitive, so Git will merge any pulled commits into the branch you are currently working in. pull automatically merges the commits without letting you review them first. If you don't closely manage your branches, you may run into frequent conflicts.
When you fetch, Git gathers any commits from the target branch that do not exist in your current branch and stores them in your local repository. However, it does not merge them with your current branch. This is particularly useful if you need to keep your repository up to date, but are working on something that might break if you update your files. To integrate the commits into your master branch, you use merge.

What is Git fork? What is the difference between fork, branch and clone?

A fork is a remote, server-side copy of a repository, distinct from the original. A fork isn't a Git concept really, it's more a political/social idea.
A clone is not a fork; a clone is a local copy of some remote repository. When you clone, you are actually copying the entire source repository, including all the history and branches.
A branch is a mechanism to handle the changes within a single repository in order to eventually merge them with the rest of code. A branch is something that is within a repository. Conceptually, it represents a thread of development.

What is the difference between Rebase and Merge?

Git Merge and Git Rebase commands are used to combine the work of multiple developers in one code.
……. Read more somewhere else!!!



Could you explain the Gitflow workflow?

How to revert previous commit in git?

What is git cherry-pick?

Can you explain what git reset does in plain English?

What is the difference between git clone, git clone --bare and git clone --mirror?

How to undo the most recent commits in Git?

You accidentally committed wrong files to Git, but haven't pushed the commit to the server yet. How can you undo those commits from the local repository?

```
$ git commit -m "Something terribly misguided"
$ git reset HEAD~                              # copied the old head to .git/ORIG_HEAD
<< edit files as necessary >>
$ git add ...
```

```
$ git commit -c ORIG_HEAD    # will open an editor, which initially contains the log message from the old commit and allows you to edit it
```

What git command do you need to use to know who changed certain lines in a specific file?

React

Why is isMounted() an anti-pattern and what is the proper solution?

The primary use case for isMounted() is to avoid calling setState() after a component has been unmounted, because it will emit a warning.

```
if (this.isMounted()) {
  this.setState({...})
}
```

Checking isMounted() before calling setState() does eliminate the warning, but it also defeats the purpose of the warning. Using isMounted() is a code smell because the only reason you would check is because you think you might be holding a reference after the component has unmounted.

An optimal solution would be to find places where setState() might be called after a component has unmounted, and fix them. Such situations most commonly occur due to callbacks, when a component is waiting for some data and gets unmounted before the data arrives. Ideally, any callbacks should be canceled in componentWillUnmount(), prior to unmounting.

Can you force a component to re-render without calling setState?

By default, when your component's state or props change, your component will re-render. If your render() method depends on some other data, you can tell React that the component needs re-rendering by calling forceUpdate().

```
component.forceUpdate(callback)
```

It is recommended to avoid all uses of forceUpdate() and only read from this.props and this.state in render().

What is the difference between super() and super(props) in React using ES6 classes?

When you want to access this.props in constructor() then you should pass props to super() method.

Using super(props):

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props)
    console.log(this.props) // { name: 'John', ... }
  }
}
```

Using super():

```
class MyComponent extends React.Component {
  constructor(props) {
    super()
    console.log(this.props) // undefined
  }
}
```

Outside constructor() both will display the same value for this.props.

What is the difference between setState() and replaceState() methods?

When you use setState() the current and previous states are merged. replaceState() throws out the current state, and replaces it with only what you provide. Usually setState() is used unless you really need to remove all previous keys for some reason. You can also set state to false/null in setState() instead of using replaceState().

What is Redux Thunk?

By default, Redux's actions are dispatched synchronously, which is a problem for any non-trivial app that needs to communicate with an external API or perform side effects. Redux also allows for middleware that sits between an action being dispatched and the action reaching the reducers.

There are two very popular middleware libraries that allow for side effects and asynchronous actions: Redux Thunk and Redux Saga. In this post, you will explore Redux Thunk.

Thunk is a programming concept where a function is used to delay the evaluation/calculation of an operation.

Redux Thunk is a middleware that lets you call action creators that return a function instead of an action object. That function receives the store's dispatch method, which is then used to dispatch regular synchronous actions inside the function's body once the asynchronous operations have been completed.

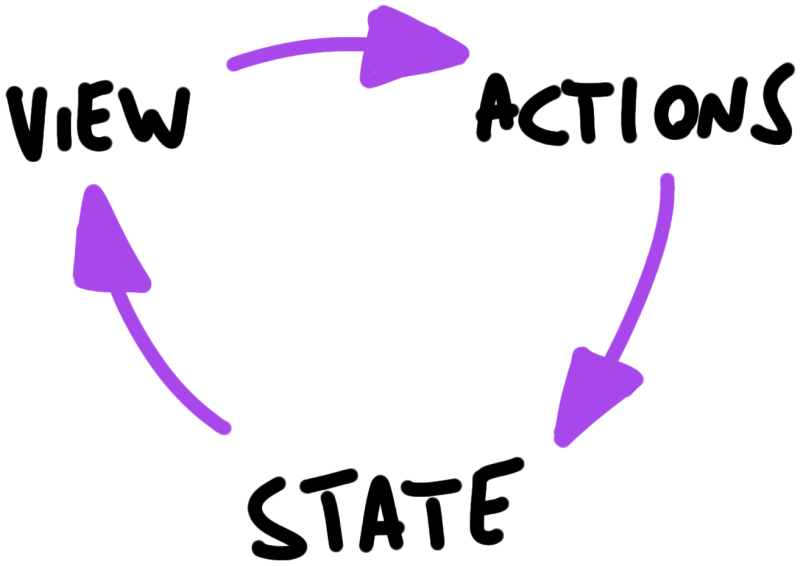
Unidirectional Data Flow in React

Unidirectional Data Flow is not a concept unique to React, but as a JavaScript developer this might be the first time you hear it.

In general this concept means that data has one, and only one, way to be transferred to other parts of the application

In React this means that:

- state is passed to the view and to child components
- actions are triggered by the view
- actions can update the state
- the state change is passed to the view and to child components



The view is a result of the application state. State can only change when actions happen. When actions happen, the state is updated.

Thanks to one-way bindings, data cannot flow in the opposite way (as would happen with two-way bindings, for example), and this has some key advantages:

- it's less error prone, as you have more control over your data
- it's easier to debug, as you know *what* is coming from *where*
- it's more efficient, as the library already knows what the boundaries are of each part of the system

A state is always owned by one Component. Any data that's affected by this state can only affect Components below it: its children.

Changing state on a Component will never affect its parent, or its siblings, or any other Component in the application: just its children.

This is the reason that the state is often moved up in the Component tree, so that it can be shared between components that need to access it.

What are *Higher-Order* components?

A higher-order component (HOC) is a function that takes a component and returns a new component. Basically, it's a pattern that is derived from React's compositional nature. We call them as "pure components" because they can accept any dynamically provided child component but they won't modify or copy any behavior from their input components.

```
const EnhancedComponent = higherOrderComponent(WrappedComponent);
```

HOC can be used for many use cases as below,

1. Code reuse, logic and bootstrap abstraction
2. Render High jacking
3. State abstraction and manipulation
4. Props manipulation
5.

What is the purpose of using super constructor with props argument?

A child class constructor cannot make use of this reference until super() method has been called. The same applies for ES6 sub-classes as well. The main reason of passing props parameter to super() call is to access this.props in your child constructors.

Passing props:

```
class MyComponent extends React.Component {
   constructor(props) {
      super(props);
      console.log(this.props);  // Prints { name: 'sudheer',age: 30 }
   }
}
```

What's the difference between a Controlled component and an Uncontrolled one in React?

This relates to stateful DOM components (form elements) and the React docs explain the difference:

- A Controlled Component is one that takes its current value through props and notifies changes through callbacks like onChange. A parent component "controls" it by handling the callback and managing its own state and passing the new values as props to the controlled component. You could also call this a "dumb component".
- An Uncontrolled Component is one that stores its own state internally, and you query the DOM using a ref to find its current value when you need it. This is a bit more like traditional HTML.

What are React Hooks?

Hooks are a new addition in React 16.8. They let you **use state and other React features without writing a class**. With Hooks, you can extract stateful logic from a component so it can be tested independently and reused. Hooks allow you to reuse stateful logic without changing your component hierarchy. This makes it easy to share Hooks among many components or with the community.

What are advantages of using React Hooks?

Primarily, hooks in general enable the extraction and reuse of stateful logic that is common across multiple components without the burden of higher order components or render props. Hooks allow to easily manipulate the state of our functional component without needing to convert them into class components.

Hooks don’t work inside classes (because they let you use React without classes). By using them, we can totally avoid using lifecycle methods, such as `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`. Instead, we will use built-in hooks like `useEffect` .

What are the lifecycle methods of ReactJS?

- componentWillMount: Executed before rendering and is used for App level configuration in your root component.
- componentDidMount: Executed after first rendering and here all AJAX requests, DOM or state updates, and set up eventListeners should occur.
- componentWillReceiveProps: Executed when particular prop updates to trigger state transitions.
- shouldComponentUpdate: Determines if the component will be updated or not. By default it returns true. If you are sure that the component doesn't need to render after state or props are updated, you can return false value. It is a great place to improve performance as it allows you to prevent a rerender if component receives new prop.

- componentWillUpdate: Executed before re-rendering the component when there are pros & state changes confirmed by shouldComponentUpdate which returns true.
- componentDidUpdate: Mostly it is used to update the DOM in response to prop or state changes.
- componentWillUnmount: It will be used to cancel any outgoing network requests, or remove all event listeners associated with the component.

What is the difference between ShadowDOM and VirtualDOM?

Virtual DOM

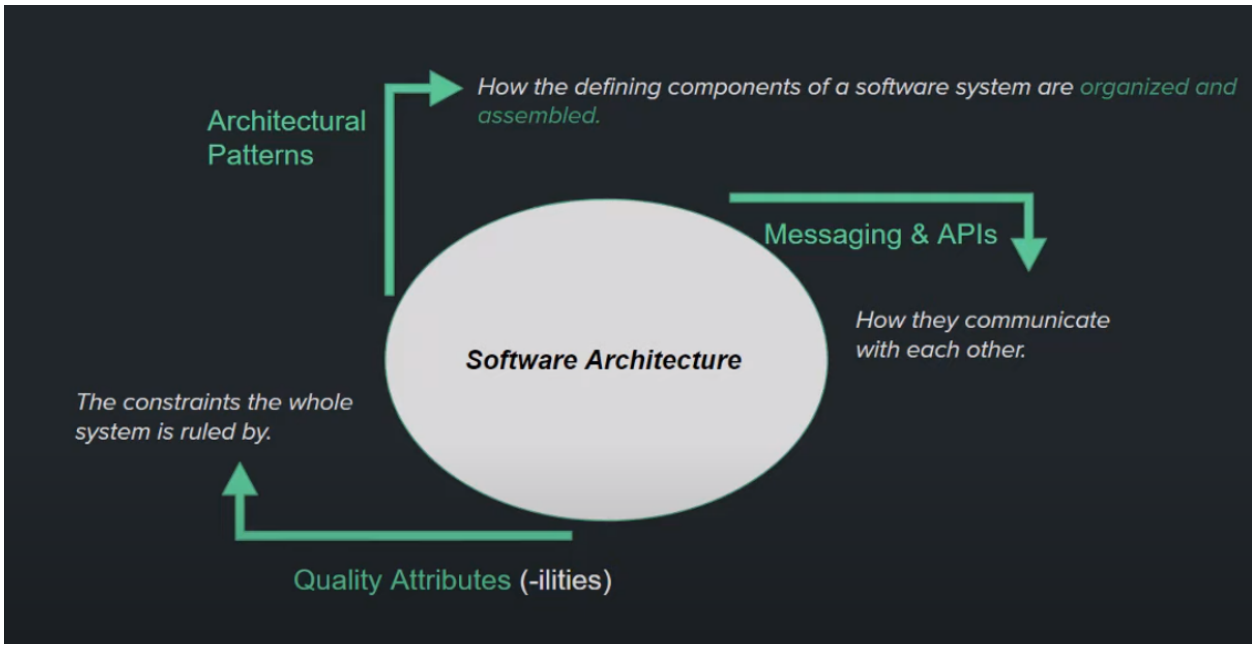
Virtual DOM is about avoiding unnecessary changes to the DOM, which are expensive performance-wise, because changes to the DOM usually cause re-rendering of the page. Virtual DOM also allows to collect several changes to be applied at once, so not every single change causes a re-render, but instead re-rendering only happens once after a set of changes was applied to the DOM.

Shadow DOM

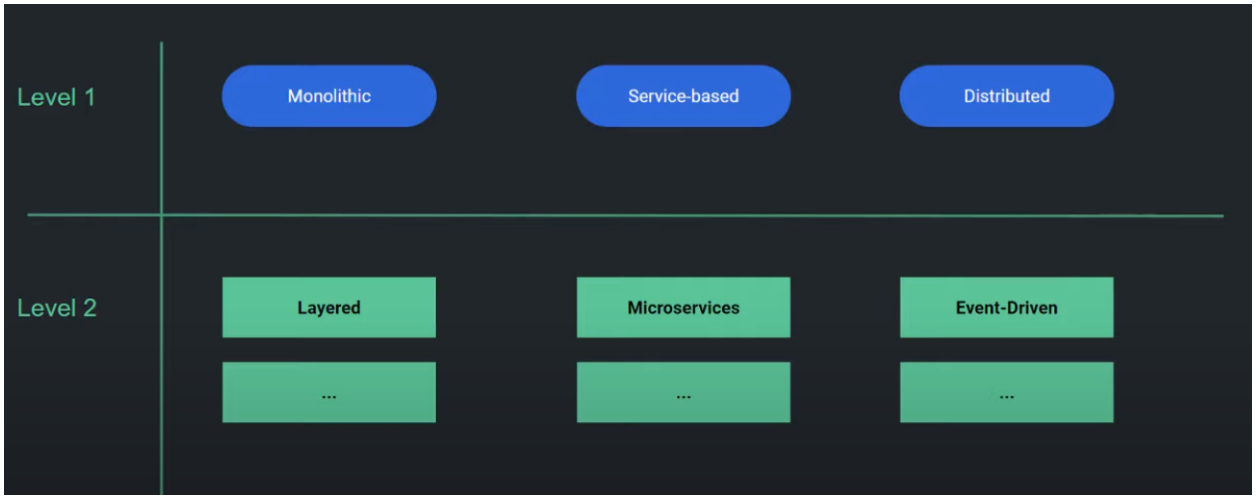
Shadow dom is mostly about encapsulation of the implementation. A single custom element can implement more-or-less complex logic combined with more-or-less complex DOM. An entire web application of arbitrary complexity can be added to a page by an import and `<body><my-app></my-app>` but also simpler reusable and composable components can be implemented as custom elements where the internal representation is hidden in the shadow DOM like `<date-picker></date-picker>`.

Software Architectural Patterns:

Software architucture is how the **defining components of a software system** are organized and assembled. How they communicate with each other. And the constraints the whole system is ruled by.



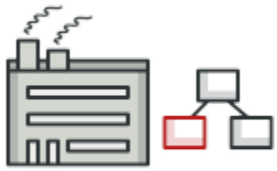
There are 3 main types:
Monolithic / service-based / distributed



Design Patterns

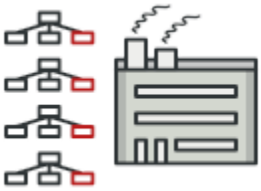
Creational Design Patterns

Creational patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.



Factory Method

Provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



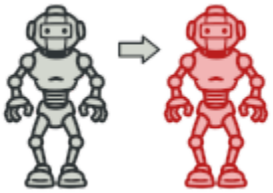
Abstract Factory

Lets you produce families of related objects without specifying their concrete classes.



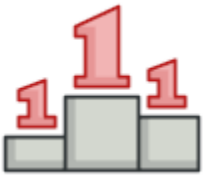
Builder

Lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.



Prototype

Lets you copy existing objects without making your code dependent on their classes.

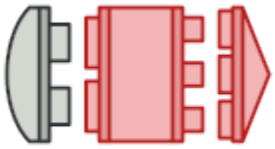


Singleton

Lets you ensure that a class has only one instance, while providing a global access point to this instance.

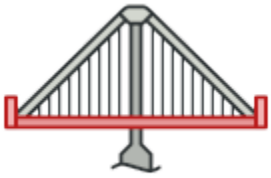
Structural Design Patterns

Structural patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient.



Adapter

Allows objects with incompatible interfaces to collaborate.



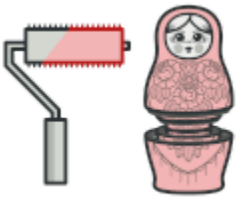
Bridge

Lets you split a large class or a set of closely related classes into two separate hierarchies—abstraction and implementation—which can be developed independently of each other.



Composite

Lets you compose objects into tree structures and then work with these structures as if they were individual objects.



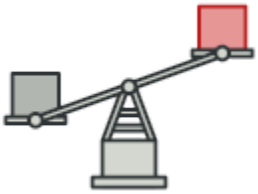
Decorator

Lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors.



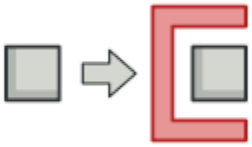
Facade

Provides a simplified interface to a library, a framework, or any other complex set of classes.



Flyweight

Lets you fit more objects into the available amount of RAM by sharing common parts of state between multiple objects instead of keeping all of the data in each object.

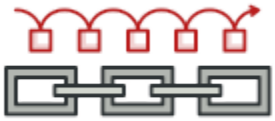


Proxy

Lets you provide a substitute or placeholder for another object. A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object.

Behavioral Design Patterns

Behavioral design patterns are concerned with algorithms and the assignment of responsibilities between objects.



Chain of Responsibility

Lets you pass requests along a chain of handlers. Upon receiving a request, each handler decides either to process the request or to pass it to the next handler in the chain.



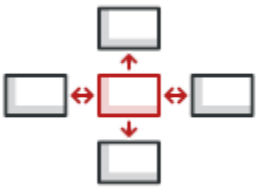
Command

Turns a request into a stand-alone object that contains all information about the request. This transformation lets you pass requests as a method arguments, delay or queue a request's execution, and support undoable operations.



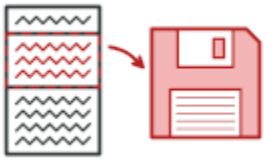
Iterator

Lets you traverse elements of a collection without exposing its underlying representation (list, stack, tree, etc.).



Mediator

Lets you reduce chaotic dependencies between objects. The pattern restricts direct communications between the objects and forces them to collaborate only via a mediator object.



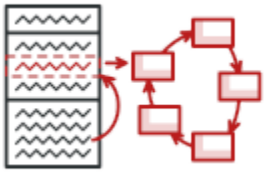
Memento

Lets you save and restore the previous state of an object without revealing the details of its implementation.



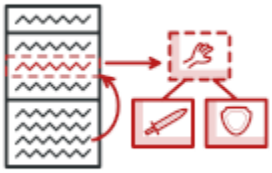
Observer

Lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.



State

Lets an object alter its behavior when its internal state changes. It appears as if the object changed its class.



Strategy

Lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.



Template Method

Defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure.



Visitor

Lets you separate algorithms from the objects on which they operate.

Microservices

Advantages of microservices:

- Distributed and loosely coupled
- Highly maintainable and testable
- Independently deployable
- Organized around business capabilities
- Owned by a small team

1. Microservices provide great technology diversity. You can mix it easily with other frameworks, libraries, and databases.
2. Microservices support fault isolation as it collects small autonomous services or processes, so a process failure should not bring the whole system down.
3. It provides excellent support for the minor and parallel team.
4. It reduces the deployment time significantly.
5. Independent deployment

What are the benefits of MicroServices?

MicroServices Architecture offers many benefits because of its characteristics.

- **Agility** - MicroServices are owned by small, independent teams who have a good understanding of business context so that they can deliver quickly. It empowers you to shorten the development time of a service.
- **Flexible Scaling** - You can scale a MicroService independently to meet the demand of application. You can control the infrastructure, cost, availability of a business functionality without affecting other services.
- **Easy Deployment** - MicroServices relies on continuous integration and continuous delivery. It allows you to push new features very quickly and you can roll-back the code anytime if anything goes wrong.
- **Technological Freedom** - You can develop a MicroService using any tool or technology which best fits your requirement. Again, MicroServices are technology agnostic.
- **Reusable Code** - MicroServices allows you to write the code or services which can work as components for other business functionality.
- **Resilience** - MicroServices are developed with failures in mind - if anything goes wrong with one service then It will not impact other services or it will not crash the entire system instead of degrading just one failure service functionality.

How to separate monolith into microservices? How do you know something needs to be a microservice?

????

What are the disadvantages of microservices?

Advantages of Microservices

- Microservices are self-contained, independent deployment module.
- The cost of scaling is comparatively less than the monolithic architecture.
- Microservices are independently manageable services. It can enable more and more services as the need arises. It minimizes the impact on existing service.
- It is possible to change or upgrade each service individually rather than upgrading in the entire application.
- Microservices allows us to develop an application which is organic (an application which latterly upgrades by adding more functions or modules) in nature.
- It enables event streaming technology to enable easy integration in comparison to heavyweight interposes communication.
- Microservices follows the single responsibility principle.
- The demanding service can be deployed on multiple servers to enhance performance.
- Less dependency and easy to test.
- Dynamic scaling.
- Faster release cycle.

Disadvantages of Microservices

- Microservices has all the associated complexities of the distributed system.
- There is a higher chance of failure during communication between different services.
- Difficult to manage a large number of services.
- The developer needs to solve the problem, such as network latency and load balancing.
- Complex testing over a distributed environment.

—----------

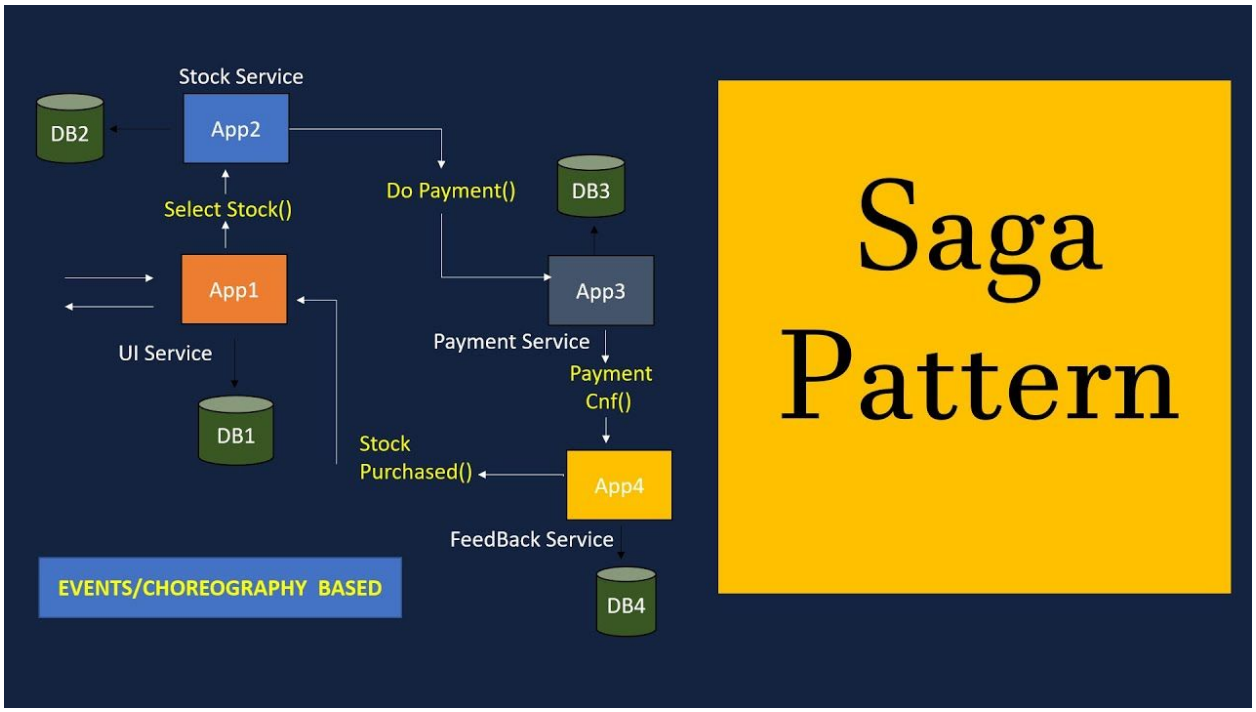
- Microservices create different types of complexity than monolithic applications for development teams.
    - First, communication between services can be complex. An application can include dozens or even hundreds of different services, and they all need to communicate securely.
    - Second, debugging becomes more challenging with microservices. With an application consisting of multiple microservices and with each microservice having its own set of logs, tracing the source of the problem can be difficult.

    - And third, while unit testing may be easier with microservices, integration testing is not. The components are distributed, and developers can't test an entire system from their individual machines.
- Interface control is even more critical.
    - Each microservice has its own API, which apps rely on to be consistent. While you can easily make changes to a microservice without impacting the external systems interacting with it, if you change the API (the interface), any application using that microservice will be affected if the change is not backwards compatible.
    - A microservices architecture model results in a large number of APIs, all crucial to the operation of the enterprise — so interface control becomes mission-critical.
- Up-front costs may be higher with microservices.
    - For microservices architecture to work for your organization, you need sufficient hosting infrastructure with security and maintenance support, *and* you need skilled development teams who understand and manage all the services.
    - If you already have these things in place, the costs involved in moving to microservices may be lower — but most enterprises that are currently running monolithic architecture will need to invest in the new infrastructure and developer resources in order to make the move.

What is Saga design pattern?

The Saga design pattern is a way to manage data consistency across microservices in distributed transaction scenarios. A saga is a sequence of transactions that updates each service and publishes a message or event to trigger the next transaction step. If a step fails, the saga executes compensating transactions that counteract the preceding transactions.

Answer 2:



This pattern, which originated in distributed systems research, is a powerful tool for managing complex, long-running processes that involve multiple microservices.

At its core, the Saga pattern is a way of coordinating the flow of data and actions between microservices. This coordination can take two forms: choreography, where each service independently communicates with others to achieve a common goal; or orchestration, where a central authority coordinates the actions of the services.

The benefits of using the Saga pattern in a .NET Web API context are many. For one, it allows for more flexible and scalable architecture, as it decouples services and allows them to be developed and deployed independently. This also makes it easier to handle failures, as the Saga pattern allows for compensating actions to be taken if a service fails or goes offline.

Additionally, the Saga pattern can help to maintain data consistency across services. In a distributed system, it is common for different services to have different views of the same data. The Saga pattern ensures that these views are kept in sync, even in the face of failures or other challenges.

Despite these benefits, there are also potential issues to be aware of when implementing the Saga pattern in .NET Web APIs. One challenge is the complexity of the coordination logic, which can become difficult to manage as the number of services and the complexity of their interactions increase.

Another potential issue is the need for careful planning and coordination between teams. In a distributed system, it is essential that all teams involved in the development and deployment of the services are aware of the overall architecture and how their individual services fit into the bigger picture.

Finally, the Saga pattern relies on the ability to undo or compensate for actions that have been taken. This requires careful design and implementation of the necessary logic, as well as robust testing to ensure that it works correctly in all scenarios.

Overall, the Saga pattern can be a powerful tool for managing complex, long-running processes in .NET Web APIs.

Coordinating the flow of data and actions between microservices, it allows for more flexible and scalable architecture, helps to maintain data consistency, and enables compensating actions to be taken in case of failures.

However, it also requires careful planning and coordination, as well as robust implementation and testing, to ensure that it is effective in practice.

In my experience, orchestrated Sagas are easier to implement and provide better decoupling. The cost of that is they are harder to test.

Answer 3:

Designing long-lived processes in a distributed environment is an interesting engineering challenge.

And a well known pattern for solving this problem is a **Saga**.

A **Saga** is a sequence of local transactions, where each local transaction updates the **Saga** state and publishes a message triggering the next step in the **Saga**.

Sagas come in two forms:

- **Orchestrated**
- **Choreographed**

With an **orchestrated Saga**, there's a central component responsible for orchestrating the individual steps.

In a **choreographed Saga**, processes work independently but coordinate with each other using events.

In this week's issue, I'll show you how to create **orchestrated Sagas** using the **Rebus** library with **RabbitMQ** for message transport.

Rebus Configuration

Rebus is a free .NET “service bus”, and it's practical for implementing asynchronous messaging-based communication between the components of an application.

Let's install the following libraries:

- **Rebus.ServiceProvider** for managing the Rebus instance
- **Rebus.RabbitMq** for RabbitMQ message transport
- **Rebus.SqlServer** for SQL Server state persistence

Inside of your **ASP .NET Core** application you will need the following configuration.

Unpacking the individual configuration steps:

- **Routing** - Configures messages to be routed by their type
- **Transport** - Configures the message transport mechanism
- **Sagas** - Configures the Saga persistence store
- **Timeouts** - Configures the timeouts persistence store

You also need to specify the queue name for sending and receiving messages.

AutoRegisterHandlersFromAssemblyOf will scan the specified assembly and automatically register the respective message handlers.

Creating The Saga With Rebus

We're going to create a **Saga** for a newsletter onboarding process.

When a user subscribes to the newsletter we want to:

- Send a welcome email immediately
- Send a follow-up email after 7 days

The first step in creating the **Saga** is defining the data model by implementing **ISagaData**. We'll keep it simple and store the **Email** for correlation, and add two flags for the distinct **steps** in our **Saga**.

Now we can define the **NewsletterOnboardingSaga** class by inheriting from the **Saga** class and implementing the **CorrelateMessages** method.

It's a best practice to use a unique value for correlation. In our case this will be the **Email**.

You also configure how the **Saga** starts with **IAmInitiatedBy**, and the individual messages the Saga handles with **IHandleMessages**.

Message Types And Naming Conventions

There are two types of messages you send in a **Saga**:

- Commands
- Events

Commands instruct the receiving component what to do.

Think: **verb, imperative.**

Events notify the Saga which process was just completed.

Think: **what happened, past tense.**

Saga Orchestration With Messages

The **NewsletterOnboardingSaga** starts by handling the **SubscribeToNewsletter** command, and publishes a **SendWelcomeEmail** command.

The **SendWelcomeEmail** command is handled by a different component, which publishes a **WelcomeEmailSent** event when it completes.

In the **WelcomeEmailSent** handler we update the **Saga** state and publish a deferred message by calling **Defer**. Rebus will persist the **SendFollowUpEmail** command, and publish it when the timeout expires.

Finally, the **SendFollowUpEmail** command is handled and we publish the **FollowUpEmailSent** event.

We update the **Saga** state again, and also call **MarkAsComplete** to complete the Saga.

Completing the **Saga** will delete it from the database.

Handling Commands With Rebus

Here's how the **SendWelcomeEmail** command handler looks like:

The important thing to highlight here is that we're using the **Reply** method to send a message back. This will reply back to the endpoint specified as the return address on the current message.

In Summary

Sagas are practical for implementing a long-lived process in a distributed system. Each business process is represented by a local transaction, and publishes a message to trigger the next step in the **Saga**.

Although **Sagas** are very powerful, they are also complicated to develop, maintain and debug.

We didn't cover a few important topics in this newsletter:

- Error handling
- Message retries
- Compensating transactions

I think you'll have some fun researching these on your own.

Take a look at the **source code for the example used in this newsletter** if you want to learn more about implementing Sagas.

Read more...
Read more 2...

What is MassTransit?

What does MassTransit add to the transport?

MassTransit is a lightweight service bus for building distributed .NET applications. The main goal is to provide a consistent, .NET friendly abstraction over the message transport. To meet this goal, MassTransit brings a lot of the application-specific logic closer to the developer in an easy to configure and understand manner.

The benefits of using MassTransit instead of the raw transport APIs and building everything from scratch, are shown below. These are just a few, and some are more significant than others. The fact that the hosting of your consumers, handlers, sagas, etc. are all managed consistently with a well documented production ready framework is the biggest advantage. You can also find numerous blog posts, podcasts, and articles written about MassTransit online.

Concurrency

MassTransit is completely asynchronous and leverages the .NET Task Parallel Library (TPL) to consume messages concurrently to achieve maximum throughput and high server utilization.

Connection management

The network is unreliable. If the application is disconnected from the message broker, MassTransit takes care of reconnecting and making sure all of the exchanges, queues, and bindings are restored.

Exception, retries, and poison messages

Your message consumers don't need to know about broker acknowledgement protocols. If your message consumer runs to completion, the message is acknowledged and removed from the queue. If you throw an exception, MassTransit uses a retry policy to redeliver the message to the consumer. If the retries are exhausted due to continued failures or other reasons, MassTransit moves the message to an error queue. If the

message did not reach a consumer due to being misrouted to the queue, the message is moved to a skipped queue.

Serialization

C# is a statically typed language, and developers work with types. RabbitMQ works with bytes. So how do you format a message over the wire? How do you handle different date/time formats (local, UTC, unspecified)? How do you deal with numbers, are they integers, longs, or decimals? MassTransit has already thought about this and implemented sensible defaults for you. And there are many serializers provided out of the box, including JSON, BSON, and XML as well as the .NET binary formatter as a last resort.

You can even protect your messages using AES-256 encryption, to keep prying eyes away and to ensure the safety of private information (to meet PCI or HIPAA requirements).

Message header and correlation

Designing a common message envelope can be a nitty-gritty affair until things stabilize. And MassTransit is already stable having been used in production since 2008. The format is well documented and has been tested with billions of messages. Furthermore, the envelope includes headers for tracking messages, including conversations, correlations, and requests. The address and host information in the envelope make it easy to build any messaging pattern.

Consumer lifecycle management

MassTransit handles consumer creation and disposal, and integrates with most major dependency injection containers using their built-in lifetime scope management. This ensures that dependencies are created and destroyed as part of the message consumption pipeline.

Routing

MassTransit provides a heavily production tested convention for using RabbitMQ exchanges to route published messages to the subscribed consumers. The structure is CPU and memory friendly, which keeps RabbitMQ happy.

Easy Unit Testing

One of the first rules of unit testing is to avoid hitting infrastructure. And RabbitMQ is just that. MassTransit includes a high-performance in-memory transport for testing every consumer using the same code that would be used in production. And the MassTransit.TestFramework NuGet package includes test harnesses that handle the setup and teardown of the bus so you can easily test your message consumers and sagas.

Sagas

Sagas are a powerful abstraction that supports message orchestration with durable state. Whether you use the original somewhat explicit syntax, or the powerful state machine syntax of Automatonymous, you can build highly available distributed workflow and coordination services easily. MassTransit supports both Entity Framework and NHibernate, using code-based mapping and migrations to simply code deployments and upgrades.

Scheduling

MassTransit has strong integration with Quartz.NET, to make it easy to schedule messages for future delivery. This brings distributed applications into the fourth dimension, making time a first-class citizen. Some incredibly powerful routing systems have been built by the authors using Quartz in combination with other MassTransit features.

There are also other scheduling providers that are supported by MassTransit, such as RabbitMQ deferred messages and Azure Service Bus scheduled enqueueing.

Monitoring

Keeping an eye on your services performance is critical, and having the right tools is a huge plus. MassTransit updates a range of performance counters as messages are processed so operations can keep an eye on message flow and compare the throughput to that of RabbitMQ.

Questions from my own interviews

Using block in c# and why it is used

When the lifetime of an IDisposable object is limited to a single method, you should declare and instantiate it in the using statement or using declaration. The using declaration calls the Dispose method on the object in the correct way when it goes out of scope. The using statement causes the object itself to go out of scope as soon as Dispose is called. Within the using block, the object is read-only and can't be modified or reassigned. A variable declared with a using declaration is read-only.

Both `using` forms ensure that Dispose (or DisposeAsync) is called even if an exception occurs within the `using` block. You can achieve the same result by putting the object inside a `try` block and then calling Dispose (or DisposeAsync) in a `finally` block; in fact, this is how the `using` statement and the `using` declaration are translated by the compiler.

—----------

Using calls Dispose() after the using-block is left, even if the code throws an exception.
So you usually use using for classes that require cleaning up after them, like IO.
So, this using block:

```
using (MyClass mine = new MyClass())
{
   mine.Action();
}
```

would do the same as:

```
MyClass mine = new MyClass();
try
{
   mine.Action();
}
Finally
{
   if (mine != null)
      mine.Dispose();
```

```
}
```

Read more…

Difference between GET and POST

GET is used to request data from a specified resource.
Note that the query string (name/value pairs) is sent in the URL of a GET request:

- GET requests can be cached
- GET requests remain in the browser history
- GET requests can be bookmarked
- GET requests should never be used when dealing with sensitive data
- GET requests have length restrictions
- GET requests are only used to request data (not modify)
- We cant pass complicated object like classes or arrays

POST is used to send data to a server to create/update a resource.
The data sent to the server with POST is stored in the request body of the HTTP request:

- POST requests are never cached
- POST requests do not remain in the browser history
- POST requests cannot be bookmarked
- POST requests have no restrictions on data length

What is the difference between IQueryable and IEnumerable?

The main difference between IEnumerable and IQueryable in C# is that IQueryable queries out-of-memory data stores, while IEnumerable queries in-memory data.
IQueryable inherits IEnumerable, so IQueryable does everything that IEnumerable does.

The main disadvantage of casting IQueryable to IEnumerable is slower performance. Since IEnumerable works on local objects, we first need to load all the objects to memory. IQueryable is better in that sense because we can pick just what we need.

IEnumerable:

1. IEnumerable is an interface that is available in the System.Collections namespace.

2. While querying the data from the database, the IEnumerable executes the “select statement” on the server-side (i.e. on the database), loads data into memory on the client-side, and then only applied the filters on the retrieved data.
3. So you need to use the IEnumerable when you need to query the data from in-memory collections like List, Array, and so on.
4. The IEnumerable is mostly used for LINQ to Object and LINQ to XML queries.
5. The IEnumerable collection is of type forward only. That means it can only move in forward, it can’t move backward and between the items.
6. IEnumerable supports deferred execution.
7. It doesn’t support custom queries.
8. The IEnumerable doesn’t support lazy loading. Hence, it is not suitable for paging like scenarios.

IQueryable:

1. The IQueryable is an interface that exists in the System.Linq Namespace.
2. While querying the data from a database, the IQueryable executes the “select query” with the applied filter on the server-side i.e. on the database, and then retrieves data.
3. So you need to use the IQueryable when you want to query the data from out-memory such as remote database, service, etc.
4. IQueryable is mostly used for LINQ to SQL and LINQ to Entities queries.
5. The collection of type IQueryable can move only forward, it can’t move backward and between the items.
6. IQueryable supports deferred execution.
7. It also supports custom queries using CreateQuery and Executes methods.
8. IQueryable supports lazy loading and hence it is suitable for paging like scenarios.

IEnumerable vs List

The main difference between IEnumerable and List in C# is that IEnumerable is an interface, while List is a concrete class. Moreover, IEnumerable is read-only and List is not. List represents the entire collection in memory, while IEnumerable provides an interface for getting the next item one-by-one (enumerating data).

Deferred execution

Difference between IEnumerable and List is clear when working with LINQ.

IEnumerable is a deferred execution while List is an immediate execution. `

IEnumerable will not execute the query until you enumerate over the data, whereas List will execute the query as soon as it's called. Deferred execution makes IEnumerable faster because it only gets the data when needed.

In the code example below, we use LINQ to query the list of numbers. Even though the LINQ `Where` returns an IEnumerable, the execution is deferred until we call `ToList()`. When you use `ToList()`, you force the compiler to reify the results right away.

IEnumerable is more efficient and faster when you only need to enumerate the data once.

The List is more efficient when you need to enumerate the data multiple times because it already has all of it in memory.

What is ajax and its advantages

acronym of Ajax is Asynchronous JavaScrip and XML
Ajax is a set of web development techniques using many web technologies on the client-side to create asynchronous web applications. With Ajax, web applications can send and retrieve data from a server asynchronously without interfering with the display and behavior of the existing page.

Pros

- Allows applications to render without data and fill data as the application gets it from the server.
- Gives platform independence to application developers
- Faster page renders

- More responsive applications
- No rerenders of whole pages are needed to update only a single area.

Cons

- Any user whose browser does not support JavaScript or XMLHttpRequest, or has this functionality disabled, will not be able to properly use pages that depend on Ajax.
- Multiple server requests need more data consumed at the client-side.
- Failure of any one request can fail the load of the whole page.
- Browsers with JS disabled will not be able to use pages using ajax.

Read more…

What is cross origin?

"CORS" stands for **C***ross*-**O***rigin* **R***esource* **S***haring*. It allows you to make requests from one website to another website in the browser, which is normally prohibited by another browser policy called the Same-Origin Policy (SOP).

At the crux of it, CORS and SOP are both browser policies that have developed in response to issues of browser security and vulnerabilities.

The specific browser vulnerability that Same Origin Policy is meant to address is called "cross-site request forgery" (CSRF, or alternatively XSRF, don't you love all these acronyms?).

The easy fix was for browsers to detect when a request is made from one website to another and prevent the response from being readable. This is the Same-Origin Policy.

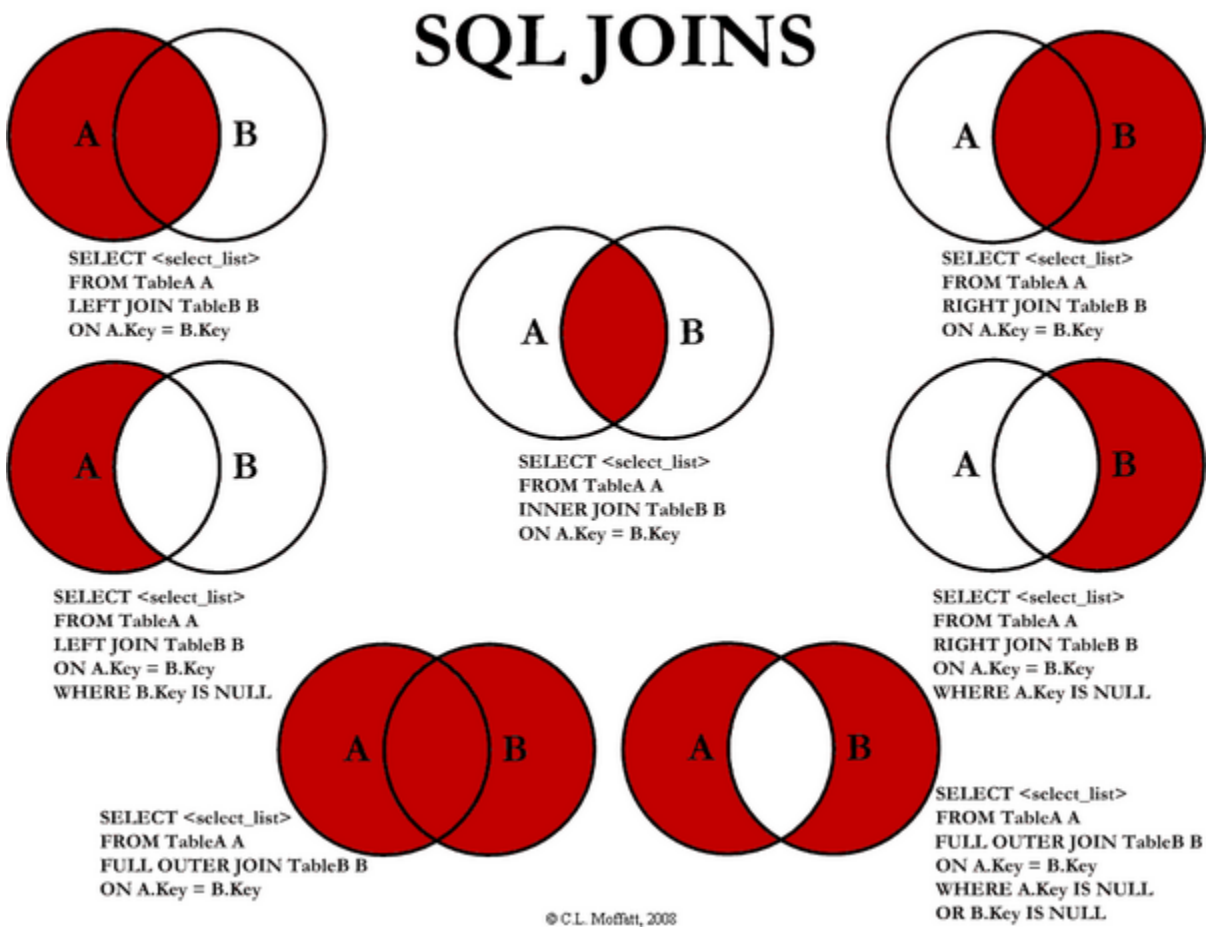
CORS allows servers to specify certain trusted 'origins' they are willing to permit requests from. Origins are defined as the combination of protocol (http or https), host (a domain like www.example.com or an

IP address) and port. Browsers which implement the CORS policy will include a HTTP header called ‘Origin’ in requests made with AJAX, including above information the value.

To instruct the browser to expose server responses to a HTTP requests from certain origin, the web server must respond to the request with an additional HTTP response header, ‘Access-Control-Allow-Origin: <origin>’. Alternatively, the web server may expose its responses to all origins by specifying a value of ‘*’, e.g. ‘Access-Control-Allow-Origin: *’.

Left join vs inner join

You’ll use INNER JOIN when you want to return only records having pair on both sides, and you’ll use LEFT JOIN when you need all records from the “left” table, no matter if they have pair in the “right” table or not.



What is index in sql?

Indexes are special lookup tables that the database search engine can use to speed up data retrieval. Simply put, an index is a pointer to data in a table. An index in a database is very similar to an index in the back of a book.
An index helps to speed up SELECT queries and WHERE clauses, but it slows down data input, with the UPDATE and the INSERT statements. Indexes can be created or dropped with no effect on the data.
SQL indexes are primarily a performance tool, so they really apply if a database gets large. SQL Server supports several types of indexes but one of the most common types is the clustered index. This type of index is automatically created with a primary key.

How do stack overflow

Infinite recursion is a common way to get a stack overflow error. To prevent - always make sure there's an exit path that will be hit. :-)

Another way to get a stack overflow (in C/C++, at least) is to declare some enormous variable on the stack.

```
char hugeArray[100000000];
```

How to prevent stack overflow?

????

What is method hiding/shadowing in c#?

C# also provides a concept to hide the methods of the base class from derived class, this concept is known as Method Hiding. It is also known as Method Shadowing. In method hiding, you can hide the implementation of the methods of a base class from the derived class using the new keyword. Or in other words, in method hiding, you can redefine the method of the base class in the derived class by using the new keyword.

Now we will see what will happen if we do not use the new keyword to hide the method of a base class from the derived class. As shown in the below example, when we do not use new keyword compiler will give run the code without giving an error, but it will give warning that if you want to hide a method then uses the new keyword.

```
prog.cs(18,14): warning CS0108: `My_Member.member()' hides inherited
member `My_Family.member()'. Use the new keyword if hiding was
intended
prog.cs(9,14): (Location of the symbol related to previous warning)
```

Read more…

Types of dependency injection?

- Constructor Injection
- Property/Setter Injection (`[Dependency]`)
- Method Injection